เรื่อง พฤติกรรมการฟังเพลงบน Spotify ในแต่ละสภาพอากาศของคนไทย

จัดทำโดย

นายธราธร ปรากฎกล้า
นายอัษฎาวุธ โหดมเทศ
นายณัฐนันท์ จารุกรกุล
นายธิติ ติยะชัยพานิช
นายรัชชนนท์ ทิพทามูล
นายศักดิ์สิทธิ์ เวชวิทยาขลัง
นายธรรมภณ แฉ่งพาที

เสนอ
ดร. สัญญ์สิริ ธารประดับ

รายงานเล่มนี้เป็นส่วนหนึ่งของรายวิชา CPE232 Data Model
ภาควิชาวิศวกรรมคอมพิวเตอร์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

สารบัญ

บทนำ 1

วัตถุประสงค์ 1

ขอบเขตงานวิจัย 1

ผลประโยชน์ที่คาดว่าจะได้รับ 1

เครื่องมือที่ใช้ในการพัฒนา 2

คำอธิบายชุดข้อมูล 3

วิธีการดำเนินงาน 6

Data preparation 7

Data Exploratory Analysis 11

Machine Learning 17

สรุปและอภิปราย 20

สรุปผลการวิจัย 20

อุปสรรคในการดำเนินงาน 20

ข้อเสนอแนะและแนวทางในการพัฒนาต่อ 20

ภาคผนวก 22

บทนำ

การฟังดนตรีเป็นหนึ่งในกิจกรรมของมนุษย์ที่มีความเป็นปัจเจกที่สุดกระนั้นแล้วการฟังดนตรีก็มักได้รับผลกระทบจากปัจจัยภายนอก ไม่ว่าจะเป็นจากอารมณ์ความรู้สึกหรือบรรยากาศในชีวิตประจำวัน ด้วยจำนวนของผู้ใช้งานที่มากของแอพลิเคชั่น Spotify เราจึงมีโอกาสในการนำข้อมูลที่ทันสมัยไปใช้งานสำหรับการวิเคราะห์ โดยเราจะหาข้อมูลเกี่ยวกับสภาพอากาศและปริมาณน้ำฝนในแต่ละวันเพื่อนำมาวิเคราะห์หาความสัมพันธ์ระหว่างสถิติการฟังเพลงและสภาพอากาศ เพื่อนำไปประยุกต์ใช้ได้ในภายภาคหน้า

วัตถุประสงค์

1. นำข้อมูลมาเข้ากระบวนการ Data Science (Process, Clean, EDA, Visualization)
2. ใช้ข้อมูลที่ผ่านกระบวนการ Data Science มาใช้เพื่อการวิเคราะห์ความสัมพันธ์ระหว่างข้อมูลของสภาพอากาศรายวันเบื้องต้นและพฤติกรรมการฟังเพลงของประชากรในประเทศไทย
3. ใช้ข้อมูลที่ผ่านกระบวนการ Data Science มาใช้ในการสร้าง Machine Learning AI

ขอบเขตงานวิจัย

โครงการนี้มีวัตถุประสงค์เพื่อวิเคราะห์ความสัมพันธ์ระหว่างสภาพอากาศและพฤติกรรมการฟังเพลงของผู้ใช้ Spotify ในประเทศไทย โดยอาศัยเทคนิคด้านวิทยาการข้อมูลและการเรียนรู้ของเครื่อง ข้อมูลที่ใช้ประกอบด้วยข้อมูลอุณหภูมิและปริมาณน้ำฝน รวมถึงข้อมูลอันดับเพลงยอดนิยมรายวันในประเทศไทยตลอดปี พ.ศ. 2566 เพื่อนำมาวิเคราะห์ว่าเงื่อนไขของสภาพอากาศต่างๆ มีผลต่อรสนิยมการฟังเพลงอย่างไร

ผลประโยชน์ที่คาดว่าจะได้รับ

- เรียนรู้การประยุกต์ใช้เทคนิคด้านวิทยาการข้อมูล เช่น การเตรียมข้อมูล การวิเคราะห์เบื้องต้น (EDA) และการสร้างภาพข้อมูลจากชุดข้อมูลจริง
- เข้าใจความสัมพันธ์ระหว่างปัจจัยทางสิ่งแวดล้อมกับพฤติกรรมมนุษย์ผ่านพฤติกรรมการเลือกเพลง
- ได้ฝึกฝนการสร้างและประเมินผลโมเดล Machine Learning จากชุดข้อมูลหลากหลายแหล่ง
- เปิดโอกาสในการนำข้อมูลสภาพอากาศมาใช้ประโยชน์ในการสร้างเพลย์ลิสต์หรือระบบแนะนำเพลงตามสภาพอากาศ

เครื่องมือใช้ในการพัฒนา

Software: Google Collaboratory with Python language and libraries & JetBrains PyCharm 2024.1 (Community Edition)

- CSV
- Datetime
- Numpy
- Pandas
- Sklearn
- Matplotlib (สำหรับการสร้าง visualization scatter plot และ box plot)

คำอธิบายชุดข้อมูล

ชุดข้อมูลที่เรานำมาใช้จัดเตรียมและวิเคราะห์สำหรับโครงงานนี้ เป็นชุดข้อมูลเกี่ยวกับอันดับของเพลงที่มียอดการฟังสูงสุดรายวันบน Spotify ของแต่ละประเทศที่ถูกปล่อยให้เป็นสาธารณะบนเว็บไซต์ kaggle [link] ซึ่งเป็นชุดข้อมูลที่รวม attribute ต่างๆ ของแต่ละเพลงจาก Spotify API และชุดข้อมูลเกี่ยวกับอุณหภูมิรายชั่วโมงที่ตรวจวัดจากระบบโทรมาตรของสถาบันสารสนเทศทรัพยากรน้ำ [link] และชุดข้อมูลเกี่ยวกับปริมาณฝนตรวจวัดโดยสถาบันสารสนเทศทรัพยากรน้ำเช่นกัน [link] โดยขอบเขตของข้อมูลที่เราเลือกใช้คือ

1. ระยะเวลาของการเก็บข้อมูลที่เราเลือกใช้จากชุดข้อมูลคือ ตั้งแต่ 1 มกราคม 2566 จนถึง 31 ธันวาคม 2566
2. ข้อมูลที่เราใช้ จะเป็นข้อมูลที่อยู่ภายในพื้นที่ประเทศไทย สำหรับชุดข้อมูลของอันดับของเพลงที่มียอดการฟังสูงสุดรายวันบน Spotify
3. ข้อมูลที่เก็บได้จากสถานีตรวจวัดอุณหภูมิภายในพื้นที่กทม. และปริมณฑล [BKK002-BKK021] สำหรับชุดข้อมูลของอุณหภูมิ
4. ข้อมูลที่เก็บได้จากสถานีตรวจวัดน้ำฝนภายในพื้นที่กทม. และปริมณฑล [17 สถานี] สำหรับชุดข้อมูลปริมาณน้ำฝน

สารสนเทศทรัพยากรน้ำ [link] และชุดข้อมูลเกี่ยวกับปริมาณฝนตรวจวัดโดยสถาบันสารสนเทศทรัพยากรน้ำเช่นกัน [link] โดยขอบเขตของข้อมูลที่เราเลือกใช้คือ

1. ระยะเวลาของการเก็บข้อมูลที่เราเลือกใช้จากชุดข้อมูลคือ ตั้งแต่ 1 มกราคม 2566 จนถึง 31 ธันวาคม 2566
2. ข้อมูลที่เราใช้ จะเป็นข้อมูลที่อยู่ภายในพื้นที่ประเทศไทย สำหรับชุดข้อมูลของอันดับของเพลงที่มียอดการฟังสูงสุดรายวันบน Spotify
3. ข้อมูลที่เก็บได้จากสถานีตรวจวัดอุณหภูมิภายในพื้นที่กทม. และปริมณฑล [BKK002-BKK021] สำหรับชุดข้อมูลของอุณหภูมิ
4. ข้อมูลที่เก็บได้จากสถานีตรวจวัดน้ำฝนภายในพื้นที่กทม. และปริมณฑล [17 สถานี] สำหรับชุดข้อมูลปริมาณน้ำฝน

ภายในชุดข้อมูลแต่ละตัว ประกอบด้วย

1. ชุดข้อมูลเกี่ยวกับอันดับของเพลงที่มียอดการฟังสูงสุดรายวันบน Spotify ของแต่ละประเทศ
    - spotify_id: unique identifier สำหรับเพลงใน database ของ Spotify
    - name: ชื่อเพลง
    - artists: ชื่อศิลปินเพลง/วงดนตรี
    - daily_rank: อันดับยอดการฟังรายวันบน top 50 ของแต่ละประเทศ

- daily_movement: การเปลี่ยนแปลงของอันดับยอดการฟังรายวันเมื่อเทียบกับวันก่อนหน้า
- Weekly_movement: การเปลี่ยนแปลงของอันดับยอดการฟังรายวันเมื่อเทียบกับสัปดาห์ก่อนหน้า
- country: ISO code (ตัวอักษรย่อชื่อประเทศ เช่น TH) ของประเทศที่ใช้จัดอันดับ
- snapshot_date: วันที่ข้อมูลถูกดึงมาจาก Spotify API
- popularity: ตัวชี้วัดความนิยมของเพลงบน Spotify
- is_explicit: ใช้บอกว่าเนื้อเพลงมีเนื้อหารุนแรงหรือไม่
- duration_ms: เวลาของเพลงในมิลลิวินาที
- album_name: ชื่ออัลบั้ม
- album_release_date: วันที่ปล่อยอัลบั้ม
- tempo: จังหวะเพลง bpm
- key: คีย์หลักของเพลง ใช้ pitch class notation ค่าอยู่ระหว่าง -1 ถึง 11 -1 = ไม่พบคีย์
- mode: 0 = minor 1 = major
- loudness: ความดังของเพลง อยู่ระหว่าง -60 ถึง 0 dB
- time_signature: เครื่องหมายกำหนดจังหวะ
- danceability: คะแนนความน่าเต้นของเพลง
- energy: คะแนนความครึกครื้นของเพลง
- speechiness: ความถี่ของบทร้อง ยิ่งเข้าใกล้ 100 มีบทร้องตลอดเพลง
- acousticness: คะแนนความเป็น acoustic ของเพลง
- instrumentalness: ความถี่ของเครื่องดนตรี (ไม่มีบทร้อง) 100 คือสูงสุด
- liveness: ค่าความแสดงสด(เสียงผู้ชม ผู้ฟัง) ใกล้ 100 แสดงว่าเป็นเพลงแสดงสด
- valence: ค่าความเป็นบวกหรือลบของเพลง สูงสุด 1.0 เพลงแนวบวก

ขนาดข้อมูล: 25 columns x 1,955,484 rows

2. ชุดข้อมูลเกี่ยวกับอุณหภูมิรายชั่วโมงที่ตรวจวัดจากระบบโทรมาตรของสถาบันสารสนเทศทรัพยากรน้ำ
   - date: วันที่วัดอุณหภูมิ
   - time: เวลาที่วัดอุณหภูมิของวันนั้นๆ
   - temp: อุณหภูมิที่วัดได้ ณ วัน-เวลานั้นๆ

ขนาดข้อมูล: 3 columns x 8760 rows x 20 files (จากสถานีที่เกี่ยวข้อง)

3. ชุดข้อมูลเกี่ยวกับปริมาณน้ำฝนรายวันที่ตรวจวัดจากระบบโทรมาตรของสถาบันสารสนเทศทรัพยากรน้ำ
   - date: วันที่วัดปริมาณฝน
   - rain: ปริมาณน้ำฝนในวันที่วัด มีหน่วยเป็นมิลลิเมตร (มม.)

   ขนาดข้อมูล: 2 columns x 365 rows x 17 files (จากสถานีที่เกี่ยวข้อง)

วิธีการดำเนินงาน

Data Preparation

เนื่องจากเป้าหมายของเราคือการวิเคราะห์และหาความสัมพันธ์ระหว่างความนิยมของเพลงเทียบกับสภาพภูมิอากาศ เราจึงจำเป็นต้อง standardize dataset โดยใช้ date เป็นเกณฑ์ สำหรับอุณหภูมิที่เป็นรายชั่วโมงจะถูกปรับเป็นค่าเฉลี่ยต่อวันแทน สำหรับปริมาณฝนมีการ clean dataset เล็กน้อย (เปลี่ยน '-' เป็น '0') ข้อมูลเพลงใน Spotify ก็จะถูกกรองมาแค่เฉพาะข้อมูลของประเทศไทย หลังจากที่เราเตรียมข้อมูลทั้ง 3 ชุดเสร็จแล้วจะนำมารวมกันโดยใช้ date เป็น key จะได้ผลลัพธ์เป็น

Exploratory Data Analyzing

เราจะนำข้อมูลมา categorize เพื่อความง่ายในการวิเคราะห์ข้อมูล ดังนี้

อุณหภูมิ

- ร้อน
- ปกติ
- เย็น

โดยข้อมูลอุณหภูมิต้องนำไปผนวกกับข้อมูลปริมาณฝนเพื่อระบุสภาพภูมิอากาศที่ชัดเจน

ปริมาณฝน

- ไม่มีฝน
- ฝนปรอยๆ
- ฝนตกหนัด

โดยเมื่อได้สภาพภูมิอากาศที่แม่นยำจะสามารถเทียบกับเพลงได้

Attribute ของเพลง

- Valence (เพลงมีไวบ์บวกแค่ไหน ให้ความรู้สึกมีความสุข)
- Energy (เพลงมีพลังงานแค่ไหน)
- Danceability (เพลงให้ความรู้สึกอยากเต้นตามมากแค่ไหน)
- Acousticness (มีความ acoustic แค่ไหน)

จากนั้นนำข้อมูลมา visualise เพื่อทำการวิเคราะห์หาความสัมพันธ์ระหว่างสภาพอากาศกับเพลงติดอันดับของแต่ละวัน โดยการใช้ boxplots หรือ scatterplots

Machine Learning

หลังจากผ่านกระบวนการเตรียมข้อมูลและการวิเคราะห์เบื้องต้น (EDA) แล้ว เราจะนำข้อมูลเหล่านี้ไปประยุกต์ใช้ในการสร้างโมเดล Machine Learning AI โดยมีเป้าหมายเพื่อ คาดการณ์สภาพอากาศของวันหนึ่งๆ จากคุณลักษณะของเพลงที่ผู้คนฟัง

สรุปและอภิปราย

เมื่อดำเนินการตามขั้นตอนต่างๆ ในกระบวนการ data science เสร็จเรียบร้อย เราจะนำ ผลลัพธ์ที่ได้มาทำการวิเคราะห์ สรุปและอภิปราย รวมถึงแนวทางในการพัฒนาต่อในอนาคต

1. Data Cleaning & Preparation

ก่อนที่เราจะเข้ากระบวนการ EDA เราจะต้องทำการเตรียม dataset ก่อน เพื่อให้เราสามารถทำงานกับ dataset ได้อย่างสะดวก

1. **การเตรียม dataset ปริมาณฝนเฉลี่ยรายวัน**
   Dataset จากระบบโทรมาตรของสถาบันสารสนเทศทรัพยากรน้ำ มีการแบ่งแยกไฟล์ตามปี เดือน และสถานี เราคำนวณค่าปริมาณฝนเฉลี่ยรายวันจากการรวมค่าในแต่ละสถานี (ในที่นี้เราใช้ข้อมูลจากสถานี BKK01-BKK17) สร้าง dataset ใหม่ที่เป็นค่าเฉลี่ยรายวันโดยรวม นอกจากนี้มีการแทนค่าบางค่าที่หายไปใน dataset (ค่าใน dataset เป็น -) ด้วยค่า 0 เนื่องจากค่าที่หายไปเหล่านี้อยู่ในช่วงของปีที่ไม่มีฝนตก ค่าเฉลี่ยของรายเดือนถ้าไม่เป็น 0 ก็ใกล้เคียง 0 เราจึงแทนค่าที่หายไปด้วยค่า 0
2. **การเตรียม dataset อุณหภูมิเฉลี่ยรายวัน**
   Dataset จากระบบโทรมาตรของสถาบันสารสนเทศทรัพยากรน้ำ มีการแบ่งแยกไฟล์ตามปี เดือน และสถานี เราทำการสร้าง dataset ใหม่ที่เป็นค่าเฉลี่ยอุณหภูมิรายวันเช่นเดียวกันกับ dataset ปริมาณฝน แต่ในกรณีนี้สถาบันฯ มีการบันทึกค่าอุณหภูมิเป็นรายชั่วโมง เพราะฉนั้นเราจึงต้องคำนวณค่าเฉลี่ยรายวันจากค่าอุณหภูมิรายชั่วโมงสำหรับแต่ละสถานีก่อน ได้ค่าเฉลี่ยรายวันแต่ละสถานี แล้วจึงมาคำนวณค่าเฉลี่ยรายวันโดยรวมอีกที
3. **การเตรียม dataset Spotify**
   Dataset Spotify top ranking by country มีความสะอาดมากแต่มีจำนวณปริมาณข้อมูลมหาศาล เพื่อให้การดำเนินงานขั้นถัดไปง่ายขึ้น เราทำการ filter ประเทศของข้อมูลเป็นประเทศไทยเท่านั้น
4. **เช็คความพร้อมของ dataset**
   ก่อนที่เราจะเข้ากระบวนการ EDA ต่อไปเราจะต้องเช็คความถูกต้องของ dataset ต่างๆ ของเรา ซึ่งทำได้โดยการใช้ python panda เปลี่ยนข้อมูลเป็น dataframe แล้วเช็คความถูกต้องด้วยฟังก์ชันต่างๆของ library นี้

เช็คข้อมูลที่หายไป (Null)

Input:

```
AVGtemp = pd.read_csv('temp.csv')
SUMrain = pd.read_csv('rain.csv')
spotify = pd.read_csv('SpotifyTH.csv')
print(" Spotify Missing values")
print(spotify.isnull().sum(),'\n')
print(" Rain Missing values")
print(SUMrain.isnull().sum(),'\n')
print(" Temperature Missing values")
print(AVGtemp.isnull().sum(),'\n')
```

Output:

```
 Spotify Missing values
spotify_id            0
name                  0
artists               0
daily_rank            0
daily_movement        0
weekly_movement       0
country               0
snapshot_date         0
popularity            0
is_explicit           0
duration_ms           0
album_name            0
album_release_date    0
danceability          0
energy                0
key                   0
loudness              0
mode                  0
speechiness           0
acousticness          0
instrumentalness      0
liveness              0
valence               0
tempo                 0
time_signature        0
dtype: int64

 Rain Missing values
date    0
rain    0
dtype: int64

 Temperature Missing values
date            0
average_temp    0
dtype: int64
```

รูปที่ 1 output จากการเช็คข้อมูลที่หายไป

เช็ครายละเอียดของ dataset

Input:

```
print(spotify.info(),'\n')
print(SUMrain.info(),'\n')
print(AVGtemp.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18261 entries, 0 to 18260
Data columns (total 25 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   spotify_id          18261 non-null  object
 1   name                18261 non-null  object
 2   artists             18261 non-null  object
 3   daily_rank          18261 non-null  int64
 4   daily_movement      18261 non-null  int64
 5   weekly_movement     18261 non-null  int64
 6   country             18261 non-null  object
 7   snapshot_date       18261 non-null  object
 8   popularity          18261 non-null  int64
 9   is_explicit         18261 non-null  bool
 10  duration_ms         18261 non-null  int64
 11  album_name          18261 non-null  object
 12  album_release_date  18261 non-null  object
 13  danceability        18261 non-null  float64
 14  energy              18261 non-null  float64
 15  key                 18261 non-null  int64
 16  loudness            18261 non-null  float64
 17  mode                18261 non-null  int64
 18  speechiness         18261 non-null  float64
 19  acousticness        18261 non-null  float64
 20  instrumentalness    18261 non-null  float64
 21  liveness            18261 non-null  float64
 22  valence             18261 non-null  float64
 23  tempo               18261 non-null  float64
 24  time_signature      18261 non-null  int64
dtypes: bool(1), float64(9), int64(8), object(7)
memory usage: 3.4+ MB
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   date    367 non-null    object
 1   rain    367 non-null    float64
dtypes: float64(1), object(1)
memory usage: 5.9+ KB
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 2 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   date          367 non-null    object
 1   average_temp  367 non-null    float64
dtypes: float64(1), object(1)
memory usage: 5.9+ KB
None
```

รูปที่ 2 output เป็นรายละเอียดของ dataset ต่างๆ

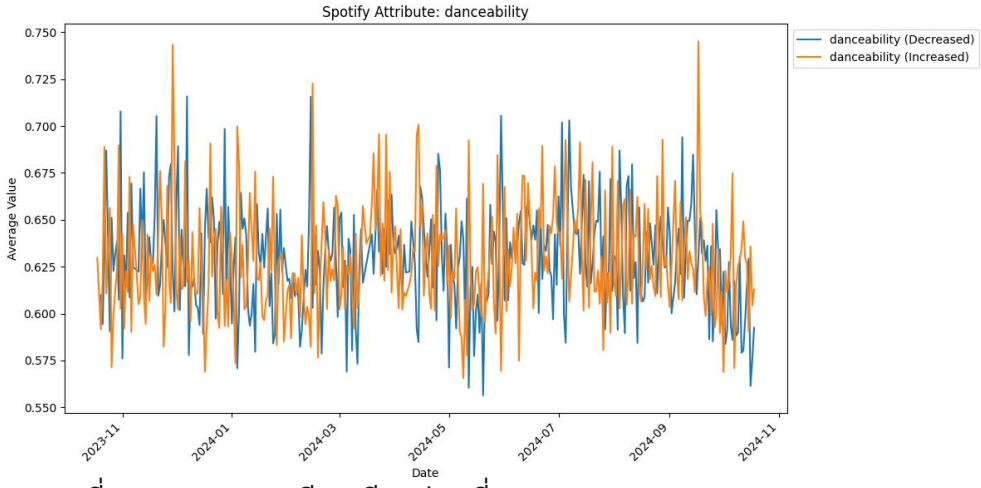
5. การเปลี่ยน type ของข้อมูล และ การ Merge dataset

ต่อไปเป็นการรวม dataset ทั้ง 3 เข้าด้วยกันเพื่อทำการ EDA
จากการเช็ครายละเอียดของข้อมูลพบว่าข้อมูลวันที่ของทั้ง 3 dataset ไม่ใช่ type datetime ที่ทำงานได้กับ dataframe เพราะฉนั้นเราจะต้องแปลงค่าให้เป็น type ที่ถูกต้องด้วยคำสั่ง .todatetime() ซึ่งเราจะต้องทำเพื่อให้ dataset ทั้ง 3 อัน merge รวมกันได้โดยไม่มีปัญหา
* การ merge dataset spotify กับ dataset
ของปริมาณฝนและอุณหภูมิทำต่อในภายหลัง (ระหว่างการทำ EDA) เมื่อต้องการใช้

```
fn = pd.merge(AVGtemp,SUMrain, on = 'date', how = 'inner')
fn['date'] = pd.to_datetime(fn['date'])
spotify['snapshot_date'] =
pd.to_datetime(spotify['snapshot_date'])
```

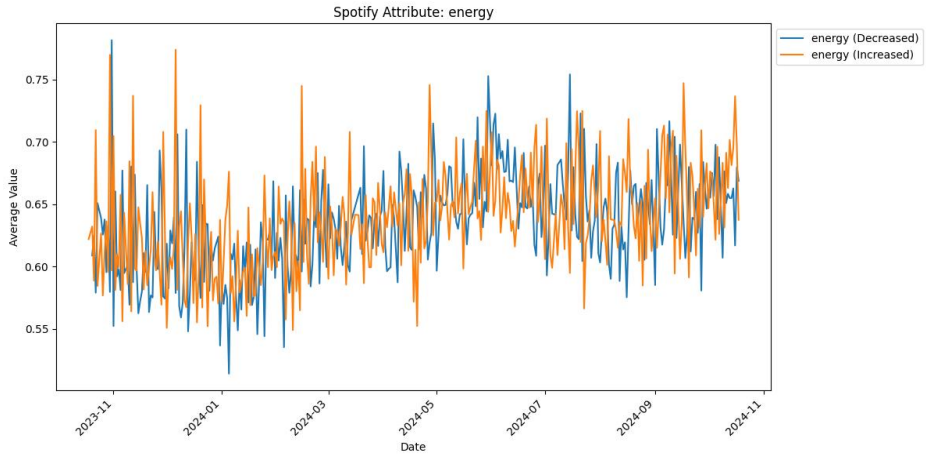
2. Exploratory Data Analysis (EDA)

ในกระบวนการ EDA เราได้ทำการ Visualise dataset ต่างๆในหัวข้อที่เราสนใจ เพื่อที่จะช่วยในการวิเคราห์หาความสัมพันธ์ระหว่างพฤติกรรมการฟังเพลงของประชกรชาวไทยกับสภาพภูมิอากาศ โดยในส่วนนี้ของรายงานเป็นการวิเคราห์ข้อมูลจากที่ได้ทำการ visualise ไว้บางส่วน สำหรับผลการ visualise ทั้งหมด สามารถดูได้ที่ภาคผนวก



รูปที่ 3 แสดง กราฟเปรียบเทียบค่าเฉลี่ยของ attribute danceability

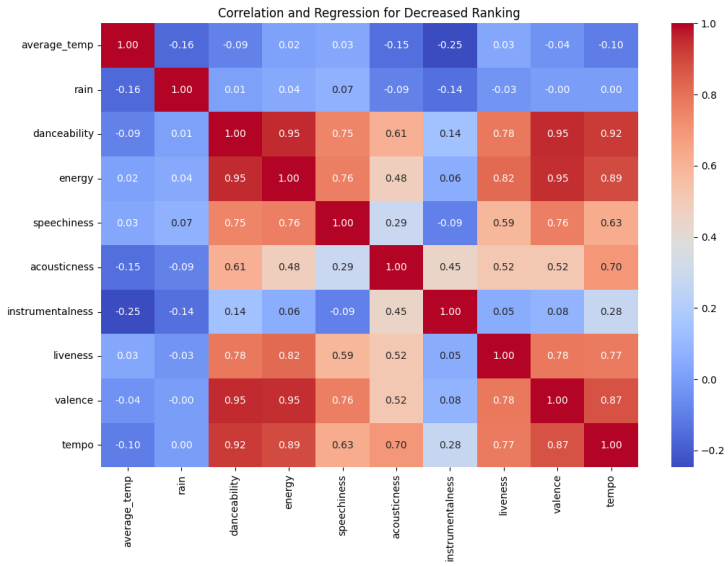
กราฟมีลักษณะที่มีการเปลี่ยนแปลงตลอดเวลาเมื่อเปรียบเทียบค่าเฉลี่ยของ **danceability** (ความสามารถในการเต้นของเพลง)ในกลุ่มของ เพลงที่อันดับเพิ่มขึ้น และ เพลงที่อันดับลดลง ซึ่งจะหมายความว่าค่าเฉลี่ย danceability แปรผันทุกวัน และไม่มีแนวโน้มที่ชัดเจนและนอกจากนี้ เพลงที่อันดับเพิ่มขึ้นและลดลง ยังมีค่าเฉลี่ย danceability ใกล้เคียงกันมากในหลายๆช่วงเวลา ซึ่งทำให้สรุปได้ว่า attribute นี้อาจจะไม่ใช่ปัจจัยหลักที่มีผลต่อการเปลี่ยนอันดับเพลง หรือไม่ออกเป็นปัจจัยที่ส่งผลต่อการเปลี่ยนอันดับของเพลงน้อยมาก



รูปที่ 4 แสดง กราฟเปรียบเทียบค่าเฉลี่ยของ attribute energy

ภาพกราฟนี้แสดงค่าเฉลี่ยของ attribute energy ของเพลงในแต่ละวัน โดยแบ่งเป็น 2 กลุ่ม เหมือนกับรูปภาพที่ 4 ซึ่งค่าเฉลี่ยของ energy
ของเพลงที่อันดับเพิ่มขึ้นและลดลงมีแนวโน้มใกล้เคียงกันมาก
และไม่สามารถระบุได้ชัดเจนจากกราฟว่า energy ส่งผลต่ออันดับเพลงโดยตรงในระยะยาว

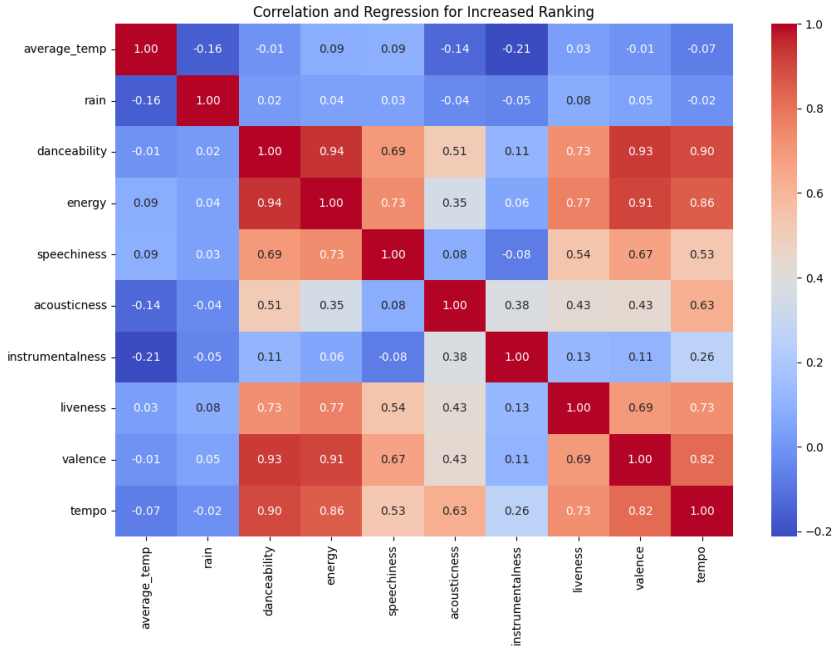
ซึ่งเราได้ทำการ visualise กราฟในลักษณะนี้สำหรับ attribute ดังนี้ danceability, energy, speechiness, acousticness, instrumentalness, liveness, valence และ tempo
สามารถดูกราฟเหล่านี้ได้ในภาคผนวก



รูปที่ 5 แสดงกราฟ Correlation Heatmap ของ dataset

จากรูปที่ 5 แสดง Correlation Heatmap ของ dataset ในกลุ่มของ เพลงที่มีอันดับลดลง ซึ่งจะแสดงความสัมพันธ์ระหว่าง attribute ต่าง ๆ ของเพลง สีใน Heatmap

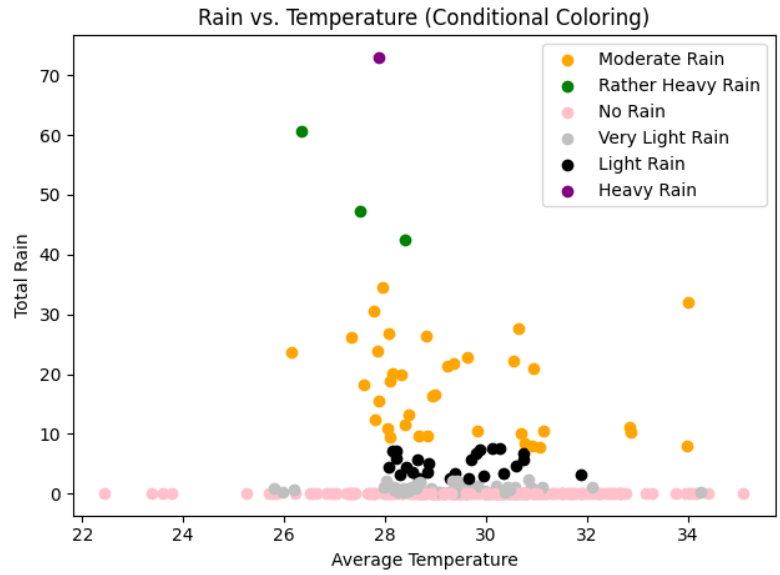
และแสดงถึงความสัมพันธ์ โดยสีโทนแดงหมายถึงความสัมพันธ์เชิงบวก
ส่วนสีโทนน้ำเงินหมายถึงความสัมพันธ์เชิงลบ
และระดับความเข้มของสีสะท้อนถึงว่าความสัมพันธ์นั้นมีความเกี่ยวข้องมากขนาดไหน
ความหลากหลายของสีบ่งชี้ว่า attribute ต่าง ๆ มีแนวโน้มที่จะไม่ขึ้นต่อกัน



รูปที่ 6 แสดงกราฟ Correlation Heatmap ของ dataset (increase ranking)

รูปที่ 6 แสดงกราฟ **Correlation Heatmap** ของชุดข้อมูล
ซึ่งแสดงความสัมพันธ์เชิงเส้นระหว่าง attribute ต่างๆ ของเพลงที่นำมาวิเคราะห์ โดยสีใน Heatmap
แสดงถึงทั้งทิศทางและระดับความแข็งแกร่งของความสัมพันธ์ระหว่างคู่ของตัวแปรแต่ละคู่
โดยโทนสีแดงแสดงถึงความสัมพันธ์เชิงบวก และโทนสีน้ำเงินแสดงถึงความสัมพันธ์เชิงลบ
ความเข้มของสีสะท้อนถึงระดับของความสัมพันธ์นั้น ๆ

จากภาพจะเห็นได้ว่า คุณลักษณะ *danceability* มีความสัมพันธ์เชิงบวกในระดับสูงกับ *energy* และ *valence* ซึ่งสะท้อนว่าเพลงที่เต้นได้ง่ายมักมีพลังงานสูงและมีอารมณ์เชิงบวก
ในขณะที่มีความสัมพันธ์เชิงลบกับ *acousticness*
ซึ่งแสดงว่าเพลงที่มีองค์ประกอบแบบอะคูสติกสูงมักจะไม่เหมาะกับการเต้นมากนัก



รูปที่ 7 แสดงกราฟ ความสัมพันธ์ระหว่างอุณหภูมิเฉลี่ยและปริมาณน้ำฝน

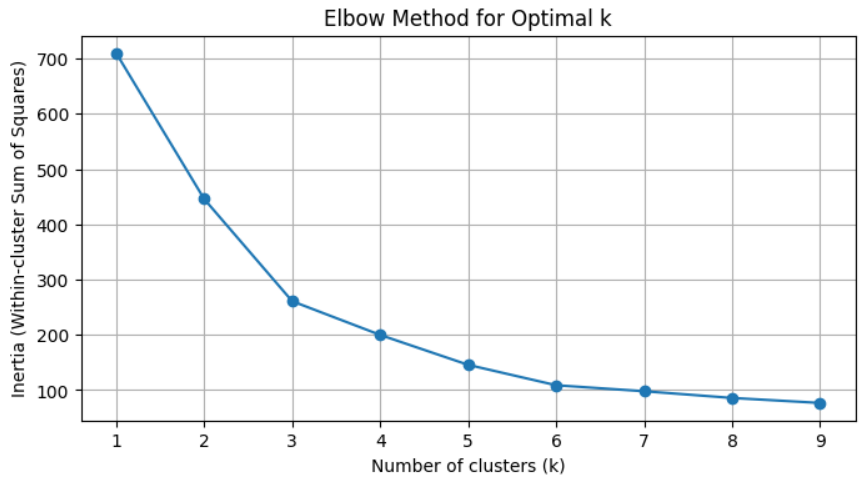
จากในกราฟดังกล่าวเราได้แยกประเภทของฝนโดยการอิงปริมาณฝนเป็นเกณฑ์ดังนี้

- ปริมาณมากกว่า 64.5 มิลลิลิตร เป็น Heavy Rain
- ปริมาณมากกว่า 35.6 มิลลิลิตร เป็น Rather Heavy Rain
- ปริมาณมากกว่า 7.6 มิลลิลิตร เป็น Moderate Rain
- ปริมาณมากกว่า 2.5 มิลลิลิตร เป็น Light Rain
- ปริมาณมากกว่า 0.1 มิลลิลิตร เป็น Very Light Rain
- ปริมาณเป็น 0 เป็น No Rain

ซึ่งมีความน่าสนใจอยู่คือ ในวันที่มีฝนตกหนัก Heavy Rain มีเพลงที่เนื้อหาเกี่ยวกับฝนบนโผล่มาบน Daily Ranking ระดับสูง

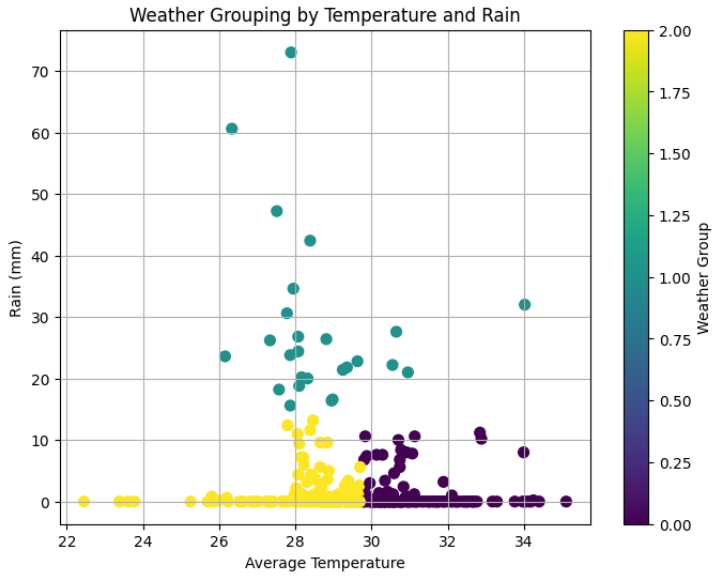
	spotify_id	name	artists	daily_rank	country	duration_ms	snapshot_date
4150	7tI8dRuH2Yc6RuoTpxo4dU	Who	Jimin	1	TH	170887	2024-07-27
4151	6vvPecFTmWxDfEJ6cYT1wa	Rockstar	LISA	2	TH	138213	2024-07-27
4152	2KslE17cAJNHTsI2MI0jb2	Standing Next to You	Jung Kook	3	TH	206019	2024-07-27
4153	2N0SPREDYqILVEFSsWF5N5	Love Me Again	V	4	TH	182293	2024-07-27
4154	4kE80JWRissrodmXuo0xYi	ดอกไม้ที่รอฝน (spring)	THE TOYS, NONT TANONT	5	TH	228000	2024-07-27

รูปที่ 8 แสดงตาราง Spotify top ranking ในวันที่ฝนตกหนัก Heavy Rain

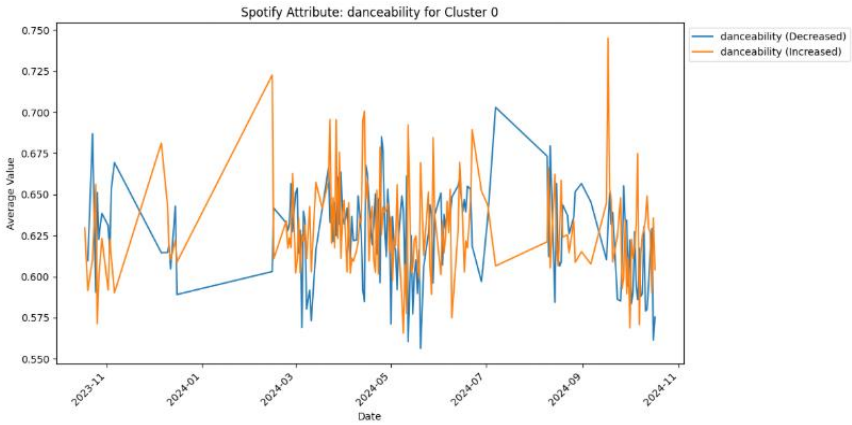


รูปที่ 9 แสดงกราฟ การหาจำนวนกลุ่มที่เหมาะสม

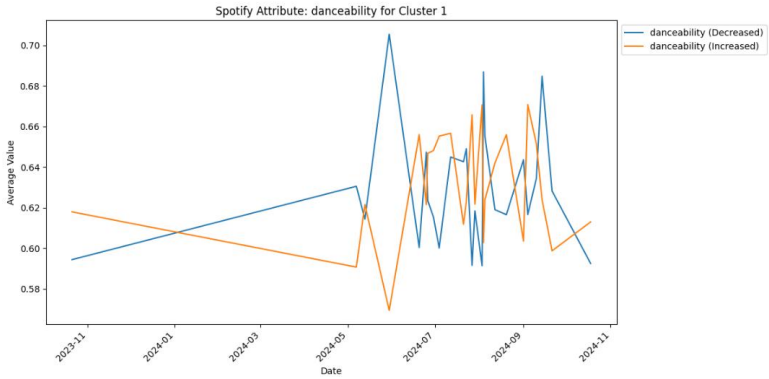
โดยการหาจำนวนการแบ่งกลุ่มที่เหมาะสมเราจะใช้อัลกอริธึม K-Means Clustering โดยใช้เทคนิคที่เรียกว่า Elbow Method โดยหลักการของ Elbow Method คือ เราจะมองหาจุดที่การลดลงของค่า Inertia เริ่มช้าลงอย่างเห็นได้ชัด จุดนั้นจะถือว่าเป็นจำนวนกลุ่มที่เหมาะสมที่สุด ตัวอย่างในรูปคือ k = 3 การทำแบบนี้อาจจะช่วยให้ Model ของเราลดโอกาสการเกิด overfitting



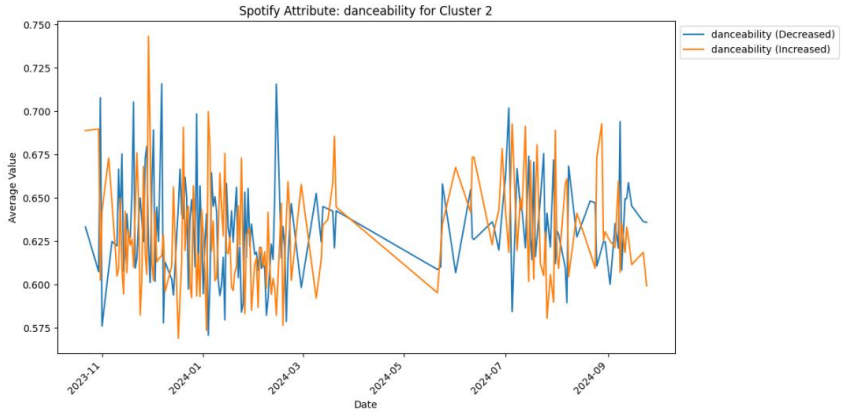
รูปที่ 10 แสดงกราฟ การแบ่งกลุ่มของข้อมูลหลังจาก ใช้ Elbow Method k = 3



รูปที่ 11 แสดง กราฟค่าเฉลี่ยของ Attribute danceability ของเพลงที่อยู่ใน Cluster 0



รูปที่ 12 แสดง กราฟค่าเฉลี่ยของ Attribute danceability ของเพลงที่อยู่ใน Cluster 1



รูปที่ 13 แสดง กราฟค่าเฉลี่ยของ Attribute danceability ของเพลงที่อยู่ใน Cluster 2

จากภาพที่ 11, 12 และ 13 จะเห็นได้ว่าแม้จะอยู่ใน cluster เดียว ค่าเฉลี่ยของ danceability ของทั้งเพลงที่มีอันดับเพิ่มขึ้นหรือลดลงกลับมีค่าที่แตกต่างกันมาก แสดงให้เห็นว่า สภาพอากาศในแต่ละวันไม่มีผลกับค่า danceability เฉลี่ยอย่างมีนัยสำคัญ

3. Model Training

ระเบียบวิธีการที่เราใช้ในการฝึกโมเดลนั้นมีดังนี้

```
song_names = train['name']  # เก็บชื่อเพลงไว้
X = train.drop(['average_temp', 'rain', 'name'], axis=1)
y = train[['average_temp', 'rain']]
```

เริ่มต้นโดยการแบ่งชุดข้อมูลที่ผ่าน Data Preparation และ Data Cleaning เป็นสองชุด ได้แก่ ชุดข้อมูลของ features (X) และ ชุดข้อมูลของ targets (y) โดยตัวแปรของแต่ละชุดข้อมูลมีดังนี้

ชุดข้อมูล X:

- danceability
- energy
- loudness
- speechiness
- acousticness
- instrumentalness
- liveness
- valence
- tempo
- is_explicit

ชุดข้อมูล y:

- average_temp
- Rain

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

จากนั้นทำการแบ่งชุดข้อมูลของ features (X) และ ชุดข้อมูลของ targets (y) เป็นชุดข้อมูลสำหรับการฝึกและชุดข้อมูลสำหรับการทดสอบ โดยใช้ฟังก์ชัน

```
base_model = RandomForestRegressor(random_state=42)
param_grid = {
   'estimator__n_estimators': [50, 100, 200],
   'estimator__max_depth': [None, 10, 20],
   'estimator__min_samples_split': [2, 5],
}

base_model = RandomForestRegressor(random_state=42)
```

โมเดลที่เราใช้ในการวิเคราะห์และพยากรณ์ข้อมูลของอุณหภูมิเฉลี่ยและปริมาณน้ำฝนเฉลี่ย ขณะที่เพลงใดๆ กำลังถูกเล่นอยู่ เป็นโมเดลประเภท Regression
เนื่องด้วยข้อมูลที่เราใช้นั้นมีลักษณะเป็นข้อมูลแบบต่อเนื่อง โดยโมเดลที่เราได้เลือกใช้คือ

- **sklearn.ensemble.RandomForestRegressor()** : เป็น regression model ที่ให้ผลลัพธ์ที่ดีกับข้อมูลที่ซับซ้อน และให้ค่า feature importance เพื่อวิเคราะห์แต่ละ predictor
- **sklearn.multioutput.MultiOutputRegressor()** : ใช้ควบคู่กับ RandomForestRegressor() เพื่อขยาย regression model ต่างๆ ให้สามารถรองรับ multi-target regression ได้ เพราะ RandomForestRegressor() สามารถรองรับตัวแปร target ได้เพียงตัวเดียว

```
grid_search = GridSearchCV(
    estimator=multioutput_model,
    param_grid=param_grid,
    scoring='r2',
    cv=3,
    verbose=1,
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
best_params = grid_search.best_params_

print("Best Parameters:", best_params)
```

ในการ Hyperparameter Tuning เราใช้ **sklearn.model_selection.GridSearchCV()** ในการย่นระยะเวลาที่ต้องใช้การค้นหาค่าเหมาะสมที่สุด โดยทำการผ่านค่า parameter ต่างๆ

```
y_pred = best_model.predict(X_test)

X_result = X_test.reset_index(drop=True)
y_result = pd.DataFrame(y_pred, columns=['average_temp', 'rain'])

rounded_pred = np.round(y_pred)
rounded_true = np.round(y_test.to_numpy())

temp_acc = accuracy_score(rounded_true[:, 0], rounded_pred[:, 0])
rain_acc = accuracy_score(rounded_true[:, 1], rounded_pred[:, 1])

accuracy = (temp_acc + rain_acc) / 2
```

และสุดท้าย นำชุดข้อมูลที่ต้องการนำมาพยากรณ์มาผ่านเข้าโมเดลที่เสร็จสมบูรณ์แล้ว และส่งออกข้อมูลของ target ที่พยากรณ์ได้ พร้อมกับการประเมินความแม่นยำของโมเดล

Result & Discussion

สรุปผลการวิจัย

หนึ่งในสิ่งที่เราต้องการจะศึกษาคือความสัมพันธ์ระหว่างสภาพอากาศกับพฤติกรรมการฟังเพลงของชาวไทย ซึ่งผลที่เราได้จากการทำการ Exploratory Data Analysis ไม่สามารถสรุปได้อย่างแน่นอนว่าสภาพอากาศส่งผลต่อพฤติกรรมการฟังเพลงของประชากรในประเทศไทยมากขนาดไหน

ในกรณีบางกรณีที่เห็นได้ชัดเป็นกรณีพิเศษ (outlier) เช่น ในวันที่มีฝนตกหนักมากเพลงที่มีเนื้อหาเกี่ยวกับฝนมีการเพิ่มความยอดนิยมขึ้น หรือในช่วงที่ฝนตกเพลงที่ฟังจะเป็นเพลง tempo และ valence ต่ำกว่าช่วงที่ฝนไม่ตก โดยค่า attribute อื่นๆ ของเพลง ไม่มีการแสดงชัดเจนว่ามีผลเท่าไหรในการเลือกฟังเพลงในแต่ละสภาพอากาศ

แต่ในกรณีทั่วไปเพลงที่ติดอันดับมักจะเป็นเพลงจากศิลปินดังที่พึ่งปล่อยมาภายในช่วงนั้นๆ หรือมีปัจจัยภายนอกที่ไม่มีอยู่ใน dataset เช่น ศิลปินมีการออกข่าวโปรโมทเพลง ช่วงวันหยุดเทศกาลอย่าง คริสต์มาส คนเราจะฟังเพลงที่มีเนื้อหาเกี่ยวกับคริสต์มาสมากขึ้น ฯลฯ

ผลจากการทำ Machine Learning ได้ผลลัพธ์ออกมามีความแม่นยำในระดับค่อนข้างสูงในส่วนของผลลัพธ์การพยากรณ์ค่าอุณหภูมิเฉลี่ย โดยผลลัพธ์ที่ได้โดยรวมจะมีค่าใกล้เคียงกับค่าที่แท้จริง และในระดับปานกลางในส่วนของผลลัพธ์การค่าปริมาณน้ำฝนเฉลี่ย โดยผลลัพธ์ที่ได้บางตัวจะมีค่าที่ต่ำหรือสูงเกินกว่าค่าที่แท้จริงมาก

อุปสรรคในการดำเนินงาน

1. เนื่องจากปริมาณอุณหภูมิของประเทศไทยโดยเฉลี่ยแล้วเป็นวงแคบ สภาพอากาศไม่มีความแตกต่างในปริมาณมาก ส่งผลให้การหาความสัมพันธ์ระหว่างพฤติกรรมการฟังเพลงของประชากรชาวไทยทำได้ยาก
2. ขอบเขตเวลาของ dataset Spotify มีให้เลือกใช้ได้น้อย เนื่องจากข้อมูลได้มาจาก Spotify API ซึ่งมีการเปลี่ยนแปลงและพัฒนาขึ้นเรื่อยๆ ทำให้ไม่มีข้อมูลในช่วงปีก่อนๆ มาใช้ได้ ซึ่งอาจจะช่วยให้ Model Training ได้ Machine Learning ที่มีความแม่นยำสูงกว่านี้
3. ปัจจัยภายนอก dataset ที่ได้เตรียมไว้ ส่งผลกระทบต่อพฤติกรรมการฟังเพลงมาก ทำให้การหาความสัมพันธ์ที่ต้องการศึกษาทำได้ยาก

ข้อเสนอแนะและแนวทางในการพัฒนาต่อ

1. อาจจะเลือกศึกษาพฤติกรรมจากประเทศอื่นเพื่อให้มีความหลากหลายในสภาพอากาศที่สังเกตได้อย่างชัดเจน เพื่อให้ง่ายต่อการ classification และช่วยให้เห็นความสัมพันธ์ต่อพฤติกรรมง่ายขึ้น

2. แทนที่จะเป็น dataset จาก Spotify อาจจะเลือกเป็น request จากผู้ฟังรายการวิทยุ เนื่องจากจะสามารถเห็นได้อย่างชัดเจนว่ามีความต้องการฟังเพลงประเภทไหนในช่วงเวลาไหน
3. เตรียม dataset ที่มีการนำปัจจัยภายนอกที่ส่งผลต่อการเลือกฟังเพลงเข้ามาพิจารณาด้วย เช่น ช่วงวันหยุดต่างๆ ในปี เทรนด์บน twitter หรือ social media อื่นๆ เป็นการช่วยกำจัดเพลงที่ไม่เกี่ยวข้องกับสภาพอากาศ หรือแยกเป็นอีกประเภทนึง

ภาคผนวก

Data preparation & EDA

Code:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

AVGtemp = pd.read_csv('temp.csv')
SUMrain = pd.read_csv('rain.csv')
spotify = pd.read_csv('SpotifyTH.csv')
print(" Spotify Missing values")
print(spotify.isnull().sum(),'\n')
print(" Rain Missing values")
print(SUMrain.isnull().sum(),'\n')
print(" Temperature Missing values")
print(AVGtemp.isnull().sum(),'\n')
```

Output:

```
 Spotify Missing values
spotify_id            0
name                  0
artists               0
daily_rank            0
daily_movement        0
weekly_movement       0
country               0
snapshot_date         0
popularity            0
is_explicit           0
duration_ms           0
album_name            0
album_release_date    0
danceability          0
energy                0
key                   0
loudness              0
mode                  0
speechiness           0
acousticness          0
instrumentalness      0
liveness              0
valence               0
tempo                 0
time_signature        0
dtype: int64

 Rain Missing values
date    0
rain    0
dtype: int64

 Temperature Missing values
date            0
average_temp    0
dtype: int64
```

Code:

```
print(spotify.info(),'\n')
print(SUMrain.info(),'\n')
print(AVGtemp.info())
```

Output:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 18261 entries, 0 to 18260
Data columns (total 25 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   spotify_id          18261 non-null  object
 1   name                18261 non-null  object
 2   artists             18261 non-null  object
 3   daily_rank          18261 non-null  int64
 4   daily_movement      18261 non-null  int64
 5   weekly_movement     18261 non-null  int64
 6   country             18261 non-null  object
 7   snapshot_date       18261 non-null  object
 8   popularity          18261 non-null  int64
 9   is_explicit         18261 non-null  bool
 10  duration_ms         18261 non-null  int64
 11  album_name          18261 non-null  object
 12  album_release_date  18261 non-null  object
 13  danceability        18261 non-null  float64
 14  energy              18261 non-null  float64
 15  key                 18261 non-null  int64
 16  loudness            18261 non-null  float64
 17  mode                18261 non-null  int64
 18  speechiness         18261 non-null  float64
 19  acousticness        18261 non-null  float64
 20  instrumentalness    18261 non-null  float64
 21  liveness            18261 non-null  float64
 22  valence             18261 non-null  float64
 23  tempo               18261 non-null  float64
 24  time_signature      18261 non-null  int64
dtypes: bool(1), float64(9), int64(8), object(7)
memory usage: 3.4+ MB
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   date    367 non-null    object
 1   rain    367 non-null    float64
dtypes: float64(1), object(1)
memory usage: 5.9+ KB
None

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 2 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   date          367 non-null    object
 1   average_temp  367 non-null    float64
dtypes: float64(1), object(1)
memory usage: 5.9+ KB
None
```

Code:

```
fn = pd.merge(AVGtemp,SUMrain, on = 'date', how = 'inner')
```

```
fn['rain'] = pd.to_numeric(fn['rain'], errors='coerce')
fn['date'] = pd.to_datetime(fn['date'])
fn
```

Output:

	date	average_temp	rain
0	2023-10-18	31.070833	7.8
1	2023-10-19	29.104167	0.2
2	2023-10-20	30.775000	8.4
3	2023-10-21	27.512500	47.2
4	2023-10-22	29.141667	0.0
...	...	...	...
362	2024-10-14	34.181818	0.0
363	2024-10-15	35.109091	0.0
364	2024-10-16	30.363636	0.2
365	2024-10-17	34.250000	0.2
366	2024-10-18	34.022222	32.0

367 rows × 3 columns

Code:

```
spotify['snapshot_date'] = pd.to_datetime(spotify['snapshot_date'])
attributes = ['danceability', 'energy', 'speechiness', 'acousticness',
'instrumentalness', 'liveness', 'valence', 'tempo']

decreased_ranking = spotify[spotify['daily_movement'] < 0]
increased_ranking = spotify[spotify['daily_movement'] > 0]
decreased_ranking_avg =
decreased_ranking.groupby('snapshot_date')[attributes].mean()
increased_ranking_avg =
increased_ranking.groupby('snapshot_date')[attributes].mean()
decreased_ranking_avg =
decreased_ranking_avg.rename_axis(index={'snapshot_date': 'date'})
increased_ranking_avg =
increased_ranking_avg.rename_axis(index={'snapshot_date': 'date'})
print("Average for Decreased Ranking:")
display(decreased_ranking_avg)
print("\nAverage for Increased Ranking:")
display(increased_ranking_avg)
```

Output:

Average for Decreased Ranking:

date	danceability	energy	speechiness	acousticness	instrumentalness	liveness	valence	tempo
2023-10-20	0.609826	0.608957	0.066096	0.327463	5.008650e-03	0.164278	0.514217	137.102130
2023-10-21	0.594400	0.624800	0.045487	0.257113	1.419487e-02	0.204013	0.483467	147.843867
2023-10-22	0.633265	0.579088	0.059009	0.338616	1.335156e-02	0.171353	0.494588	131.699206
2023-10-23	0.686909	0.650818	0.101755	0.241566	9.573791e-03	0.169209	0.548364	127.373818
2023-10-25	0.590550	0.638900	0.051105	0.301241	9.578690e-03	0.216060	0.486950	136.721700
...	...	...	...	...	...	...	...	...
2024-10-14	0.615187	0.654937	0.048469	0.205995	4.443884e-03	0.159875	0.457737	121.182875
2024-10-15	0.629312	0.662625	0.053556	0.225386	5.631250e-07	0.149050	0.557500	109.766688
2024-10-16	0.561450	0.616900	0.065615	0.290375	3.213500e-06	0.181795	0.458500	135.940350
2024-10-17	0.575357	0.679643	0.051050	0.198557	7.305914e-04	0.180671	0.493129	137.391000
2024-10-18	0.592500	0.668714	0.073407	0.284464	5.541286e-05	0.177879	0.503286	130.863714

351 rows × 8 columns

Average for Increased Ranking:

date	danceability	energy	speechiness	acousticness	instrumentalness	liveness	valence	tempo
2023-10-18	0.629714	0.621939	0.064220	0.308918	0.012336	0.176563	0.513102	134.718694
2023-10-20	0.591583	0.632083	0.052400	0.361120	0.010617	0.249567	0.449000	130.832417
2023-10-21	0.617913	0.588696	0.047583	0.355131	0.016380	0.168074	0.457957	128.497391
2023-10-22	0.688833	0.709417	0.090808	0.218911	0.010685	0.195775	0.592833	122.515583
2023-10-23	0.610733	0.584467	0.046533	0.346492	0.015829	0.191463	0.478033	131.958967
...	...	...	...	...	...	...	...	...
2024-10-14	0.626700	0.681300	0.083710	0.197655	0.000003	0.202815	0.578200	118.136150
2024-10-15	0.590389	0.695222	0.087094	0.259178	0.000797	0.220139	0.452711	130.556722
2024-10-16	0.635778	0.736500	0.063683	0.167567	0.000608	0.165167	0.566211	113.000333
2024-10-17	0.604286	0.696905	0.081695	0.258456	0.000037	0.143829	0.496952	119.901667
2024-10-18	0.612957	0.637478	0.058109	0.283621	0.002645	0.167278	0.495261	116.785652

355 rows × 8 columns

Code:

```
decreased_ranking_avg = pd.merge(fn, decreased_ranking_avg, on='date',
how='left')
increased_ranking_avg = pd.merge(fn, increased_ranking_avg, on='date',
how='left')
decreased_ranking_avg = decreased_ranking_avg.fillna(0)
increased_ranking_avg = increased_ranking_avg.fillna(0)
print("Average for Decreased Ranking:")
display(decreased_ranking_avg)
print("\nAverage for Increased Ranking:")
display(increased_ranking_avg)
```

Output:

Average for Decreased Ranking:

	date	average_temp	rain	danceability	energy	speechiness	acousticness	instrumentalness	liveness	valence	tempo
0	2023-10-18	31.070833	7.8	0.000000	0.000000	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000
1	2023-10-19	29.104167	0.2	0.000000	0.000000	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000
2	2023-10-20	30.775000	8.4	0.609826	0.608957	0.066096	0.327463	5.008650e-03	0.164278	0.514217	137.102130
3	2023-10-21	27.512500	47.2	0.594400	0.624800	0.045487	0.257113	1.419487e-02	0.204013	0.483467	147.843867
4	2023-10-22	29.141667	0.0	0.633265	0.579088	0.059009	0.338616	1.335156e-02	0.171353	0.494588	131.699206
...	...	...	...	...	...	...	...	...	...	...	...
362	2024-10-14	34.181818	0.0	0.615187	0.654937	0.048469	0.205995	4.443884e-03	0.159875	0.457737	121.182875
363	2024-10-15	35.109091	0.0	0.629312	0.662625	0.053556	0.225386	5.631250e-07	0.149050	0.557500	109.766688
364	2024-10-16	30.363636	0.2	0.561450	0.616900	0.065615	0.290375	3.213500e-06	0.181795	0.458500	135.940350
365	2024-10-17	34.250000	0.2	0.575357	0.679643	0.051050	0.198557	7.305914e-04	0.180671	0.493129	137.391000
366	2024-10-18	34.022222	32.0	0.592500	0.668714	0.073407	0.284464	5.541286e-05	0.177879	0.503286	130.863714

367 rows × 11 columns

Average for Increased Ranking:

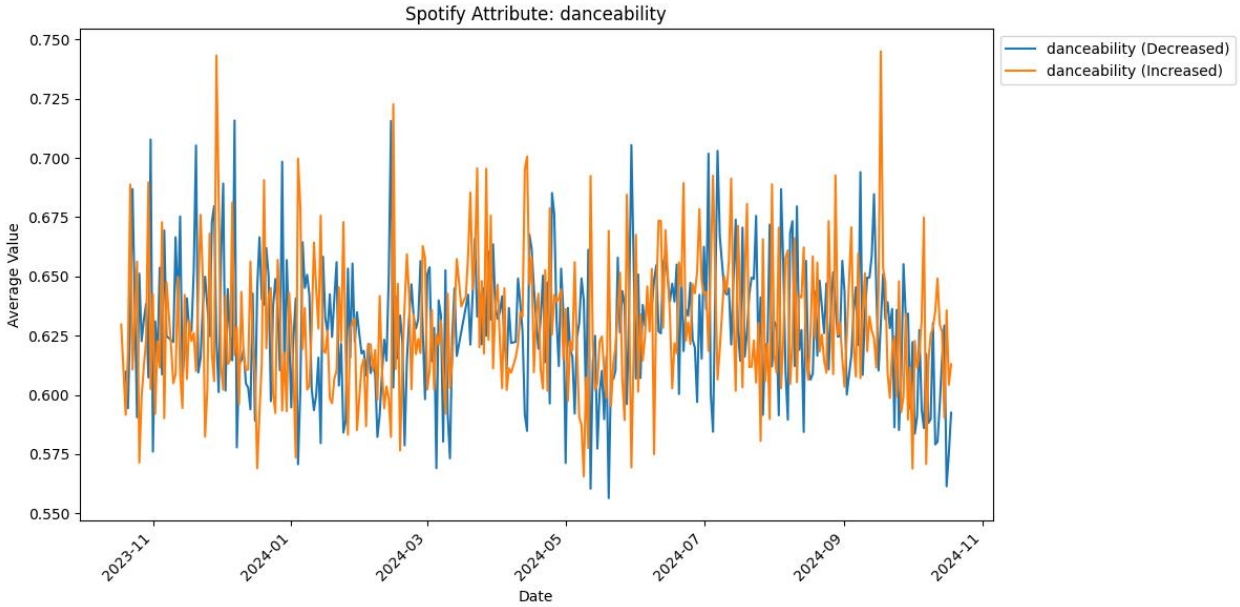
	date	average_temp	rain	danceability	energy	speechiness	acousticness	instrumentalness	liveness	valence	tempo
0	2023-10-18	31.070833	7.8	0.629714	0.621939	0.064220	0.308918	0.012336	0.176563	0.513102	134.718694
1	2023-10-19	29.104167	0.2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
2	2023-10-20	30.775000	8.4	0.591583	0.632083	0.052400	0.361120	0.010617	0.249567	0.449000	130.832417
3	2023-10-21	27.512500	47.2	0.617913	0.588696	0.047583	0.355131	0.016380	0.168074	0.457957	128.497391
4	2023-10-22	29.141667	0.0	0.688833	0.709417	0.090808	0.218911	0.010685	0.195775	0.592833	122.515583
...	...	...	...	...	...	...	...	...	...	...	...
362	2024-10-14	34.181818	0.0	0.626700	0.681300	0.083710	0.197655	0.000003	0.202815	0.578200	118.136150
363	2024-10-15	35.109091	0.0	0.590389	0.695222	0.087094	0.259178	0.000797	0.220139	0.452711	130.556722
364	2024-10-16	30.363636	0.2	0.635778	0.736500	0.063683	0.167567	0.000608	0.165167	0.566211	113.000333
365	2024-10-17	34.250000	0.2	0.604286	0.696905	0.081695	0.258456	0.000037	0.143829	0.496952	119.901667
366	2024-10-18	34.022222	32.0	0.612957	0.637478	0.058109	0.283621	0.002645	0.167278	0.495261	116.785652

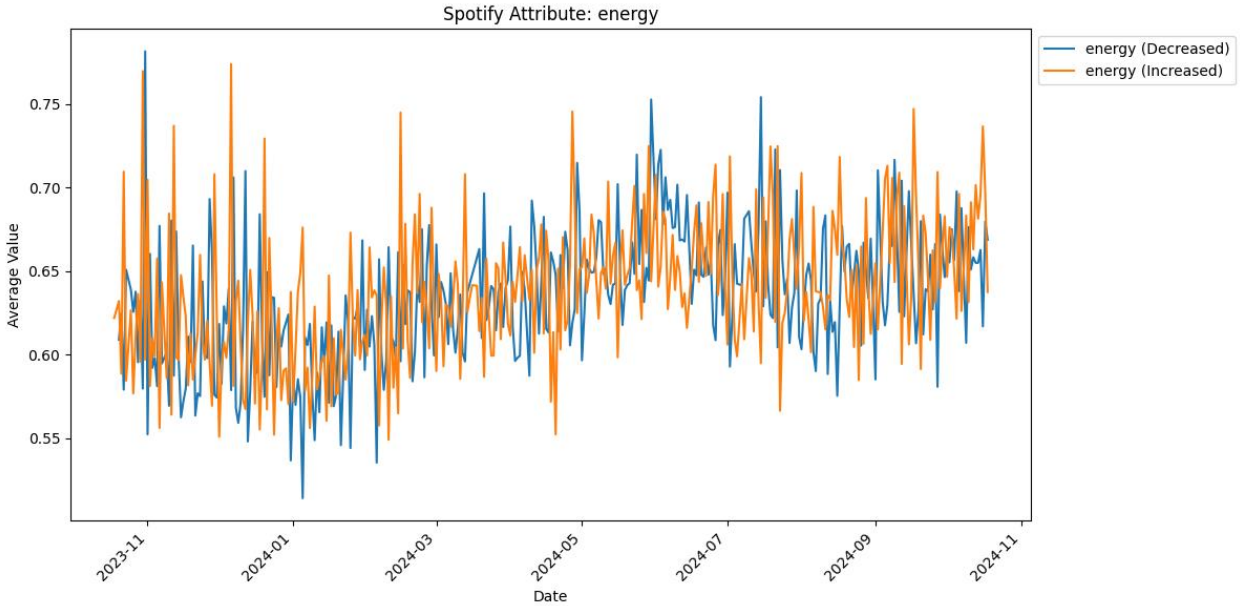
367 rows × 11 columns

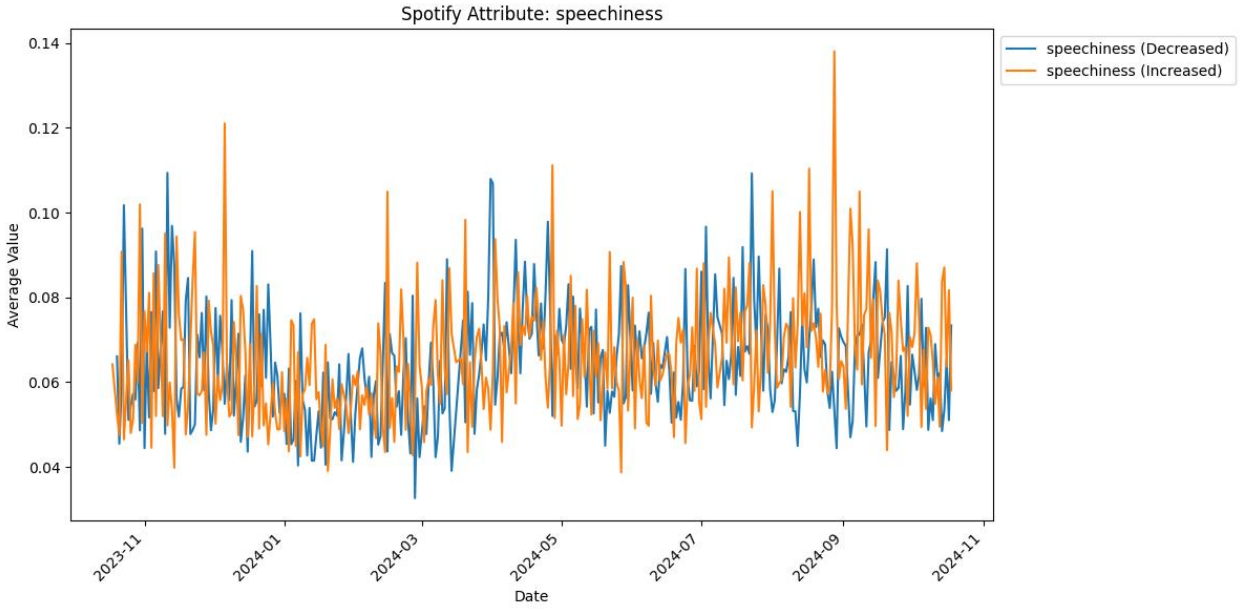
Code:

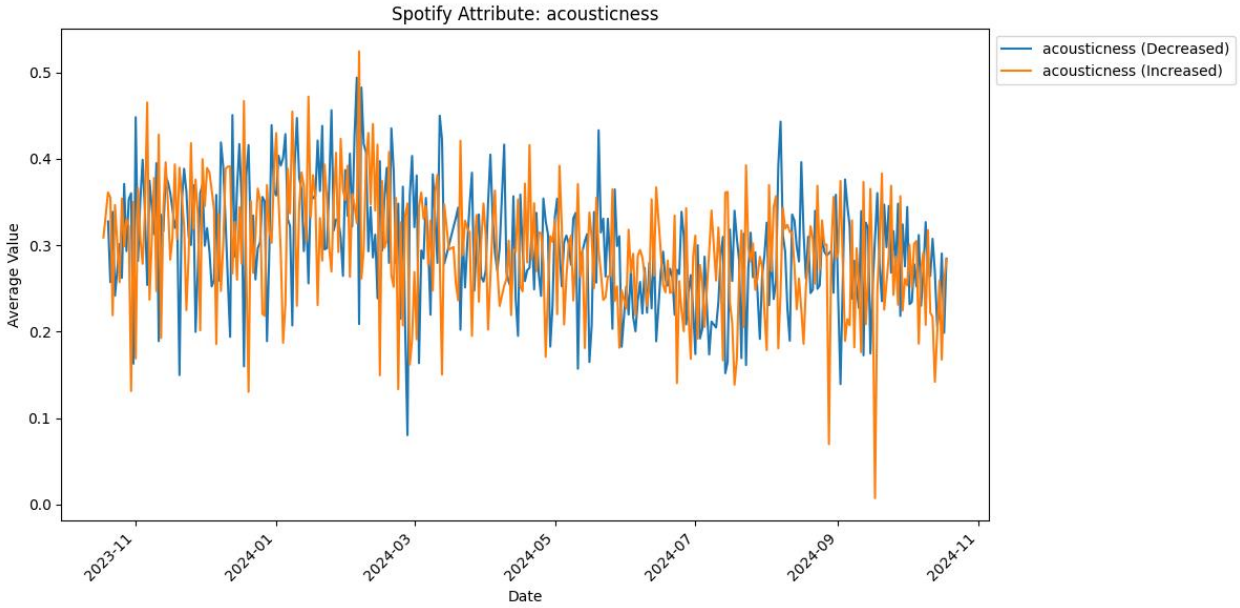
```
import matplotlib.pyplot as plt

# Assuming 'decreased_ranking_avg', 'increased_ranking_avg', and
'attributes' from your previous code

for attribute in attributes:  # Iterate through each Spotify attribute
    fig, ax = plt.subplots(figsize=(12, 6))  # Create a new figure and
axes for each attribute

    # Filter data for the current attribute, skipping zero values
    decreased_data_to_plot =
decreased_ranking_avg[decreased_ranking_avg[attribute] != 0]
    increased_data_to_plot =
increased_ranking_avg[increased_ranking_avg[attribute] != 0]

    # Plot the attribute for decreased and increased rankings
    ax.plot(decreased_data_to_plot['date'],
decreased_data_to_plot[attribute], label=f'{attribute} (Decreased)')
    ax.plot(increased_data_to_plot['date'],
increased_data_to_plot[attribute], label=f'{attribute} (Increased)')

    # Customize the plot
    ax.set_xlabel('Date')
    ax.set_ylabel('Average Value')
    ax.set_title(f'Spotify Attribute: {attribute}')  # Include attribute
in title
    ax.legend(loc='upper left', bbox_to_anchor=(1, 1))
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()
```

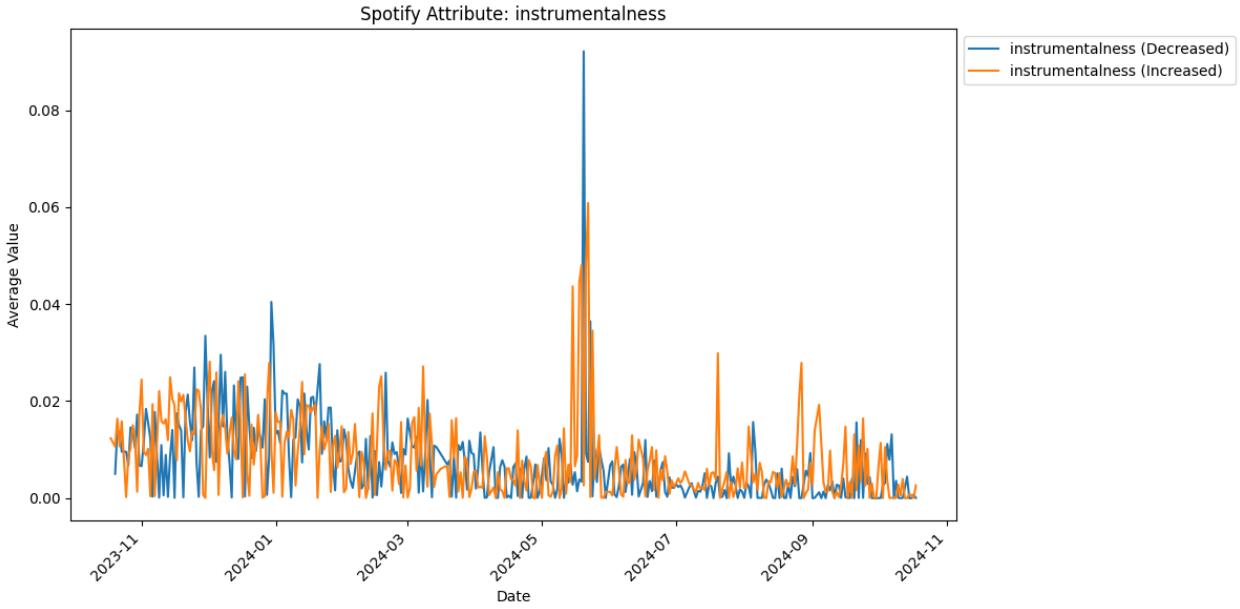
Output:

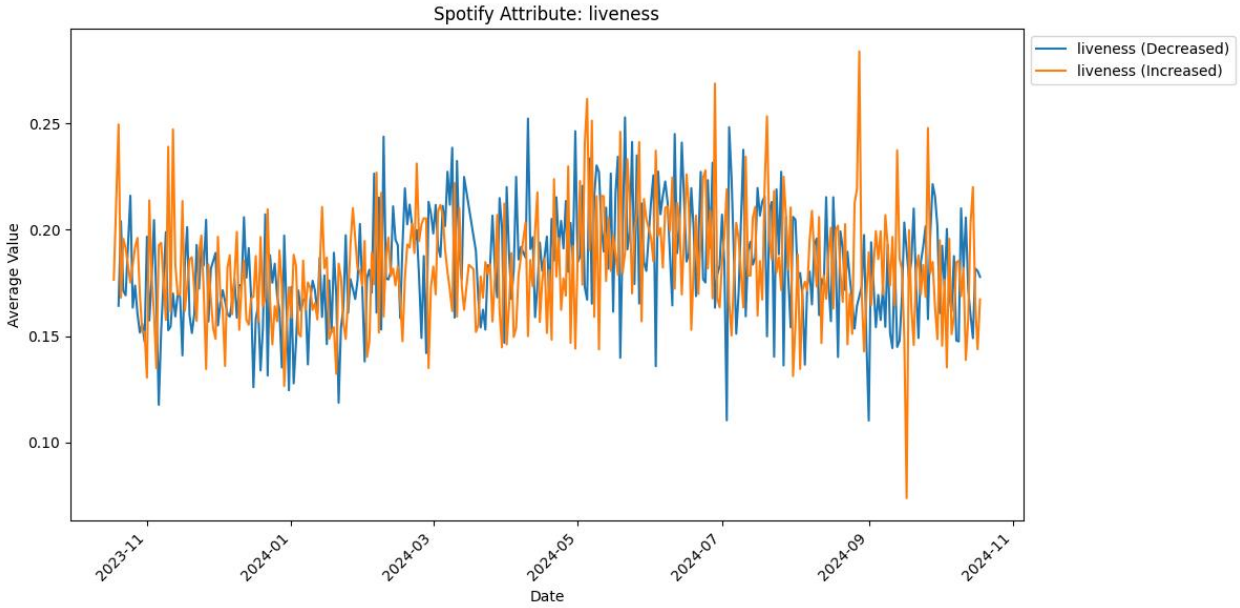

Spotify Attribute: danceability
danceability (Decreased)
danceability (Increased)
Average Value
0.750
0.725
0.700
0.675
0.650
0.625
0.600
0.575
0.550
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

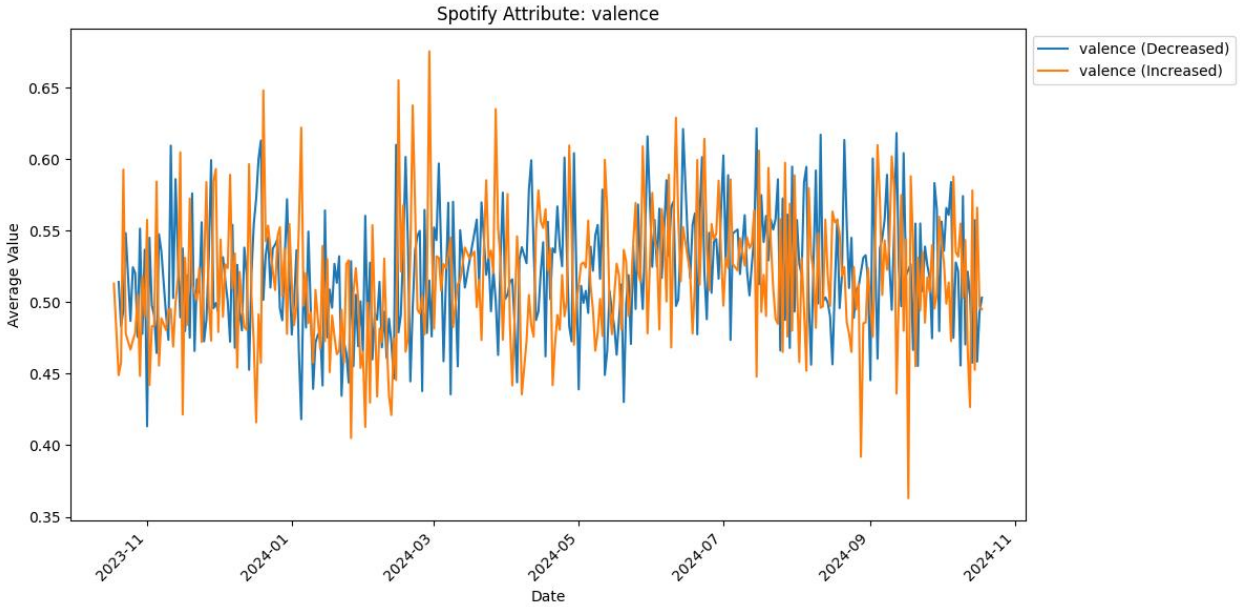

Spotify Attribute: energy
energy (Decreased)
energy (Increased)
Average Value
0.75
0.70
0.65
0.60
0.55
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

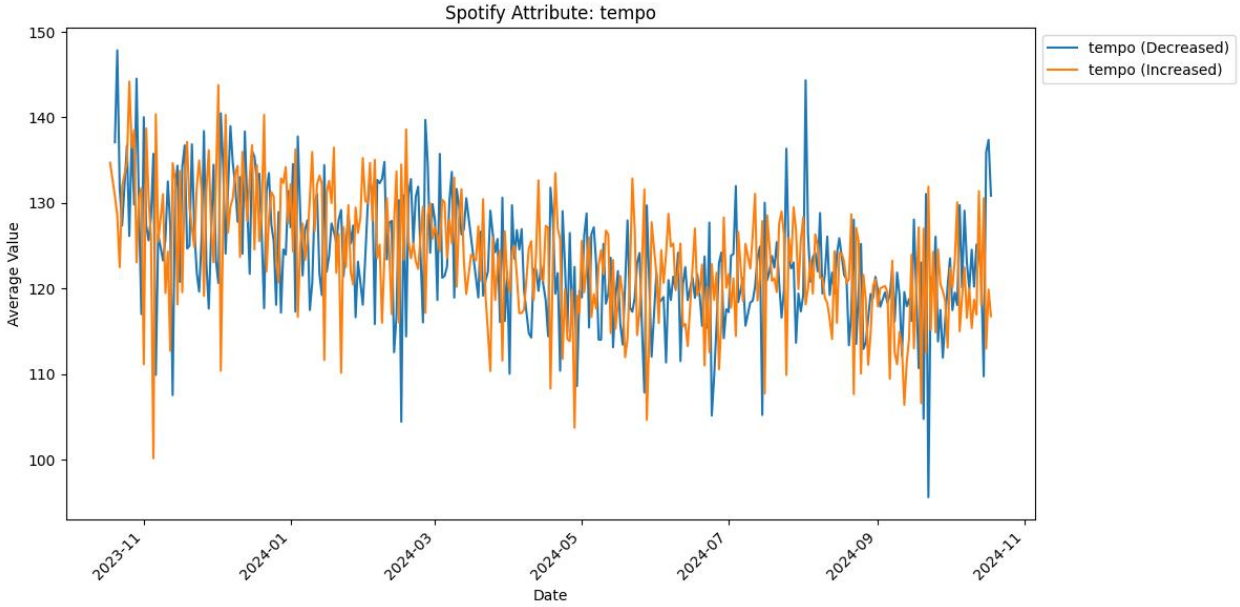

Spotify Attribute: speechiness
speechiness (Decreased)
speechiness (Increased)
Average Value
0.14
0.12
0.10
0.08
0.06
0.04
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date


Spotify Attribute: acousticness
acousticness (Decreased)
acousticness (Increased)
Average Value
0.5
0.4
0.3
0.2
0.1
0.0
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date


Spotify Attribute: instrumentalness
instrumentalness (Decreased)
instrumentalness (Increased)
Average Value
0.00
0.02
0.04
0.06
0.08
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date


Spotify Attribute: liveness
liveness (Decreased)
liveness (Increased)
Average Value
0.10
0.15
0.20
0.25
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date


Spotify Attribute: valence
valence (Decreased)
valence (Increased)
Average Value
0.65
0.60
0.55
0.50
0.45
0.40
0.35
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

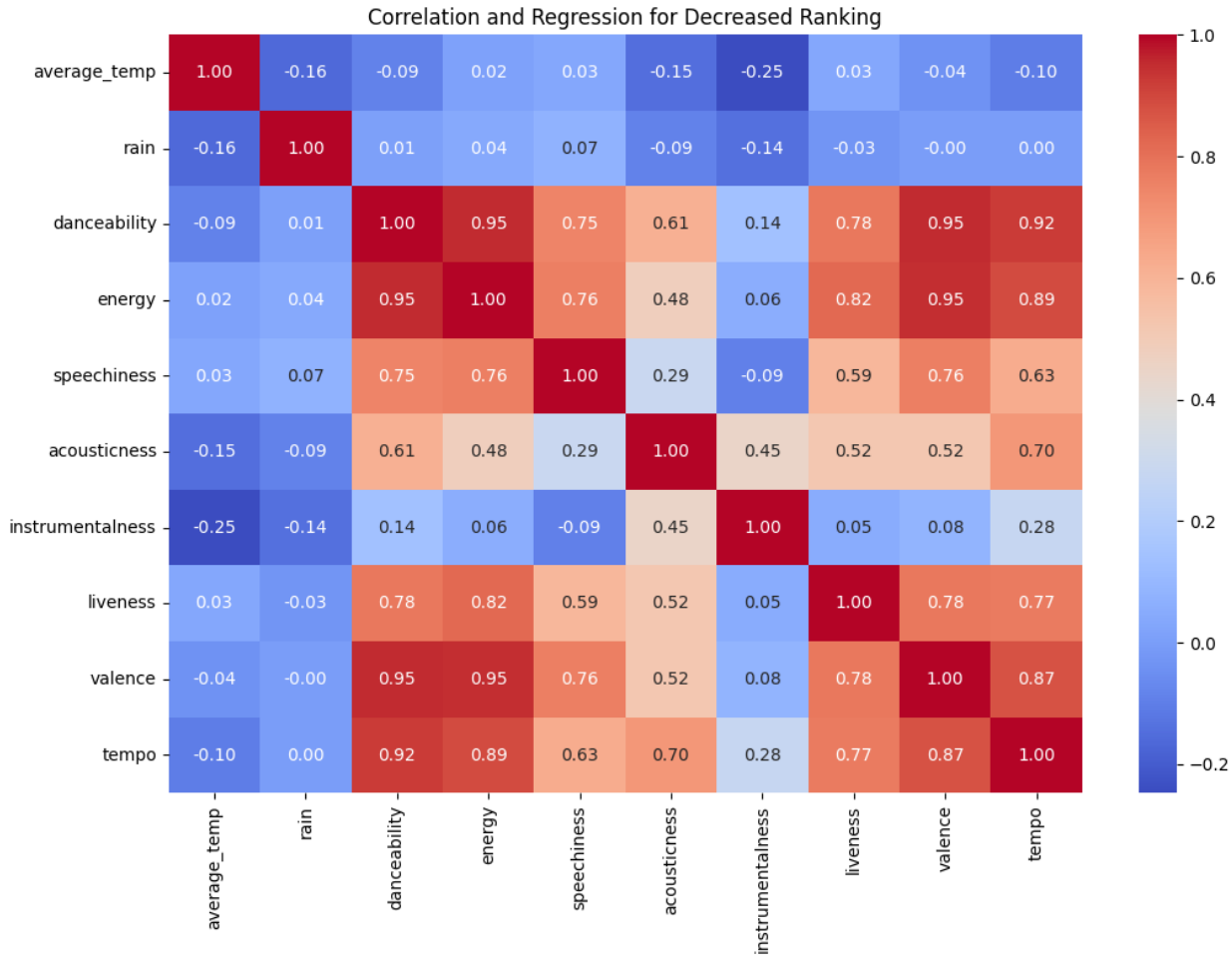

Spotify Attribute: tempo
tempo (Decreased)
tempo (Increased)
Average Value
150
140
130
120
110
100
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

Code:

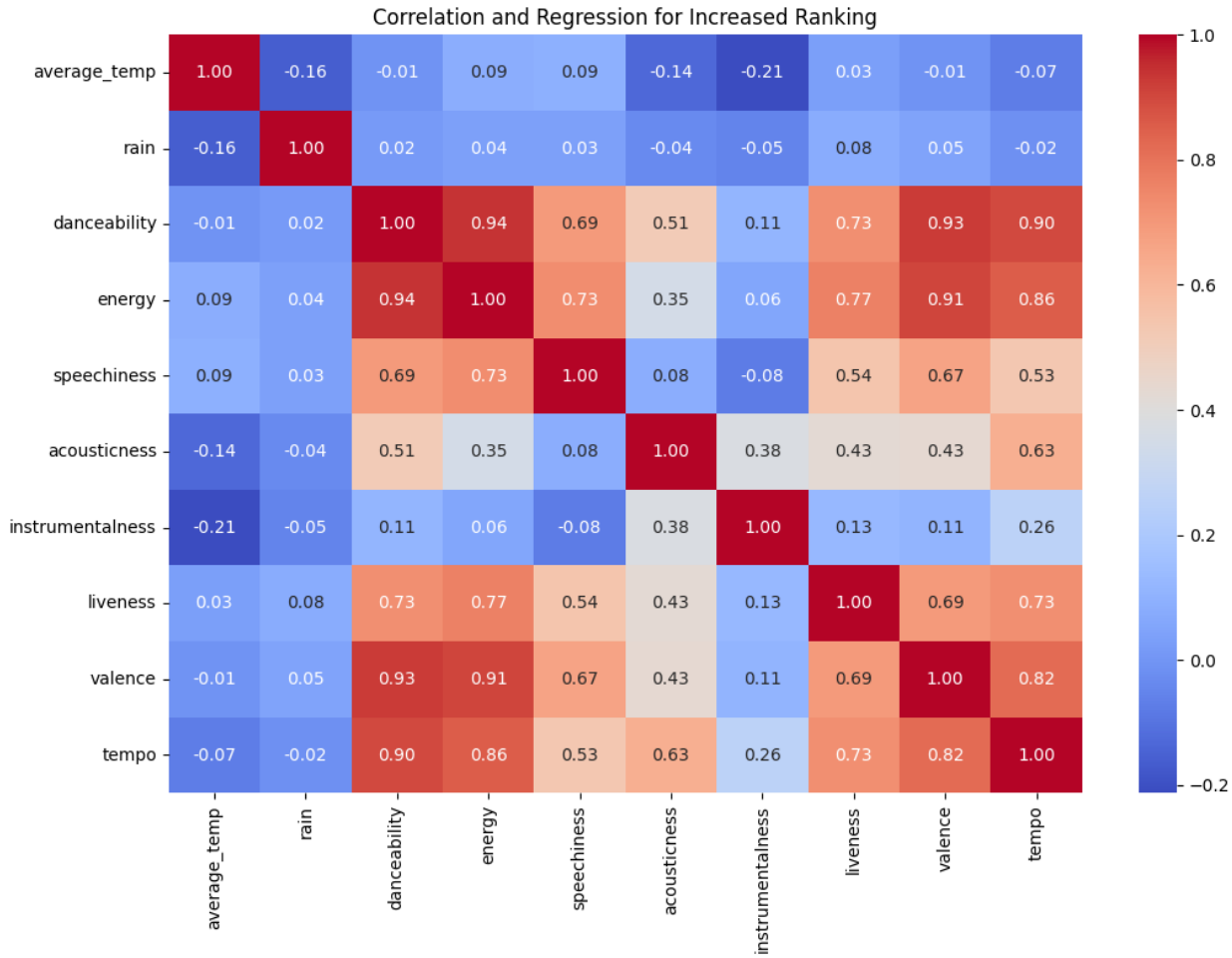
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler

# Assuming 'decreased_ranking_avg' and 'increased_ranking_avg' from your
previous code

# Function to perform analysis for a given DataFrame
def analyze_spotify_weather(df, title):
    # Correlation heatmap
    spotify_attributes = ['danceability', 'energy', 'speechiness',
'acousticness',
                          'instrumentalness', 'liveness', 'valence',
'tempo']
    corr_matrix = df[['average_temp', 'rain'] + spotify_attributes].corr()

    plt.figure(figsize=(12, 8))
    sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt=".2f")
    plt.title(title)
    plt.show()

    # Multiple Linear Regression
    X = df[['average_temp', 'rain']]
    y = df[spotify_attributes]

    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)

    model = LinearRegression()
    model.fit(X_scaled, y)

    print("Linear Regression Coefficients:")
    for i, attr in enumerate(spotify_attributes):
        print(f"  {attr}:")
        print(f"    Temperature: {model.coef_[i][0]:.4f}")
        print(f"    Rain: {model.coef_[i][1]:.4f}")
```

```
# Analyze for decreased ranking
analyze_spotify_weather(decreased_ranking_avg, 'Correlation and Regression
for Decreased Ranking')

# Analyze for increased ranking
analyze_spotify_weather(increased_ranking_avg, 'Correlation and Regression
for Increased Ranking')
```

Output:




```
Linear Regression Coefficients:
  danceability:
    Temperature: -0.0119
    Rain: -0.0009
  energy:
    Temperature: 0.0031
    Rain: 0.0062
  speechiness:
    Temperature: 0.0008
    Rain: 0.0015
  acousticness:
    Temperature: -0.0151
    Rain: -0.0105
  instrumentalness:
    Temperature: -0.0024
    Rain: -0.0016
  liveness:
    Temperature: 0.0013
    Rain: -0.0011
  valence:
    Temperature: -0.0050
    Rain: -0.0010
  tempo:
    Temperature: -2.7470
    Rain: -0.4172
```

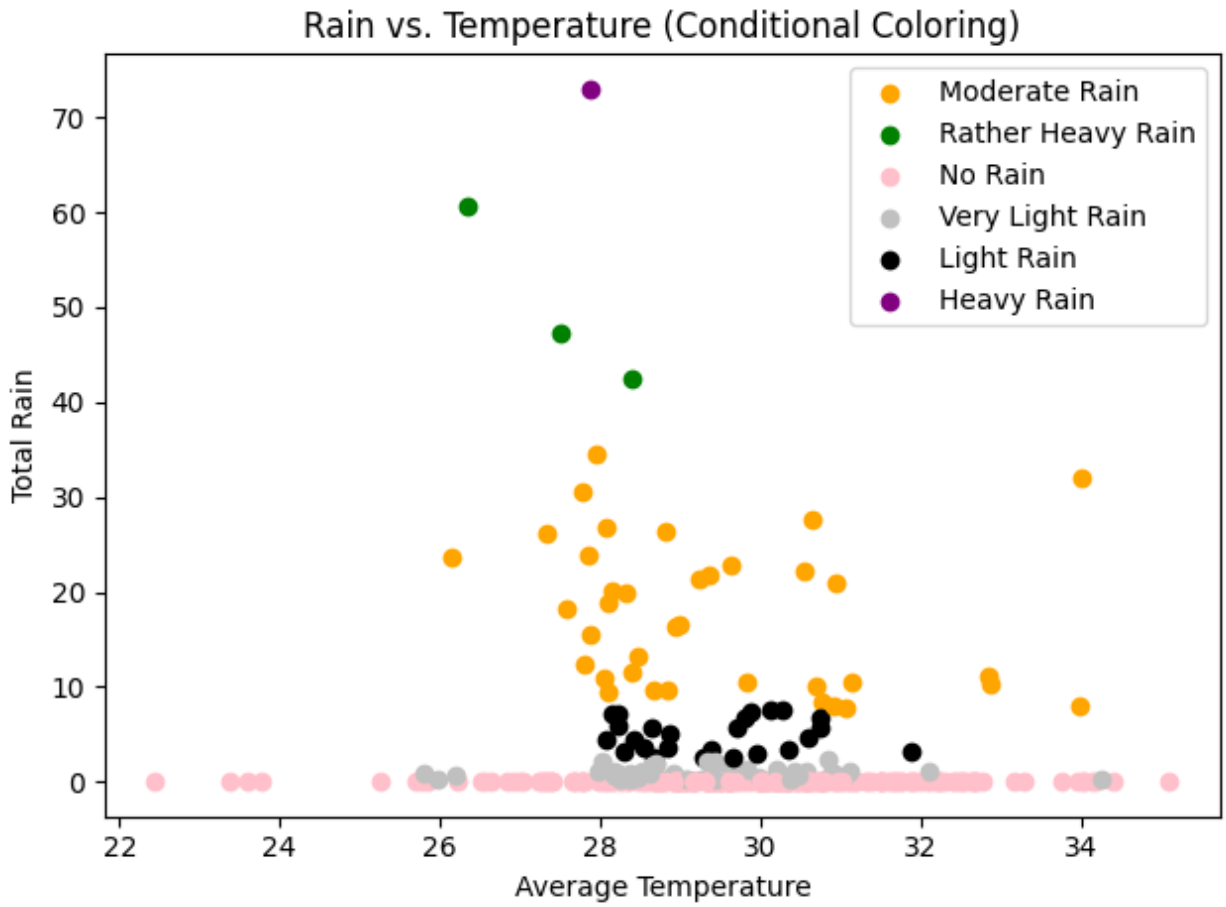



```
Linear Regression Coefficients:
  danceability:
    Temperature: -0.0005
    Rain: 0.0018
  energy:
    Temperature: 0.0113
    Rain: 0.0061
  speechiness:
    Temperature: 0.0019
    Rain: 0.0009
  acousticness:
    Temperature: -0.0128
    Rain: -0.0053
  instrumentalness:
    Temperature: -0.0020
    Rain: -0.0008
  liveness:
    Temperature: 0.0019
    Rain: 0.0037
  valence:
    Temperature: 0.0003
    Rain: 0.0048
  tempo:
    Temperature: -1.8393
    Rain: -0.7067
```

Code:

```
for index, row in fn.iterrows():
    if row['rain'] > 64.5:
        plt.scatter(row['average_temp'], row['rain'], color='purple',
label='Heavy Rain')
    elif row['rain'] > 35.6:
        plt.scatter(row['average_temp'], row['rain'], color='green',
label='Rather Heavy Rain')
    elif row['rain'] > 7.6:
        plt.scatter(row['average_temp'], row['rain'], color='orange',
label='Moderate Rain')
    elif row['rain'] > 2.5:
        plt.scatter(row['average_temp'], row['rain'], color='black',
label='Light Rain')
    elif row['rain'] > 0.1:
        plt.scatter(row['average_temp'], row['rain'], color='silver',
label='Very Light Rain')
    else:
        plt.scatter(row['average_temp'], row['rain'], color='pink',
label='No Rain')

plt.xlabel("Average Temperature")
plt.ylabel("Total Rain")
plt.title("Rain vs. Temperature (Conditional Coloring)")

handles, labels = plt.gca().get_legend_handles_labels()
by_label = dict(zip(labels, handles))
plt.legend(by_label.values(), by_label.keys())

plt.tight_layout()
plt.show()
```

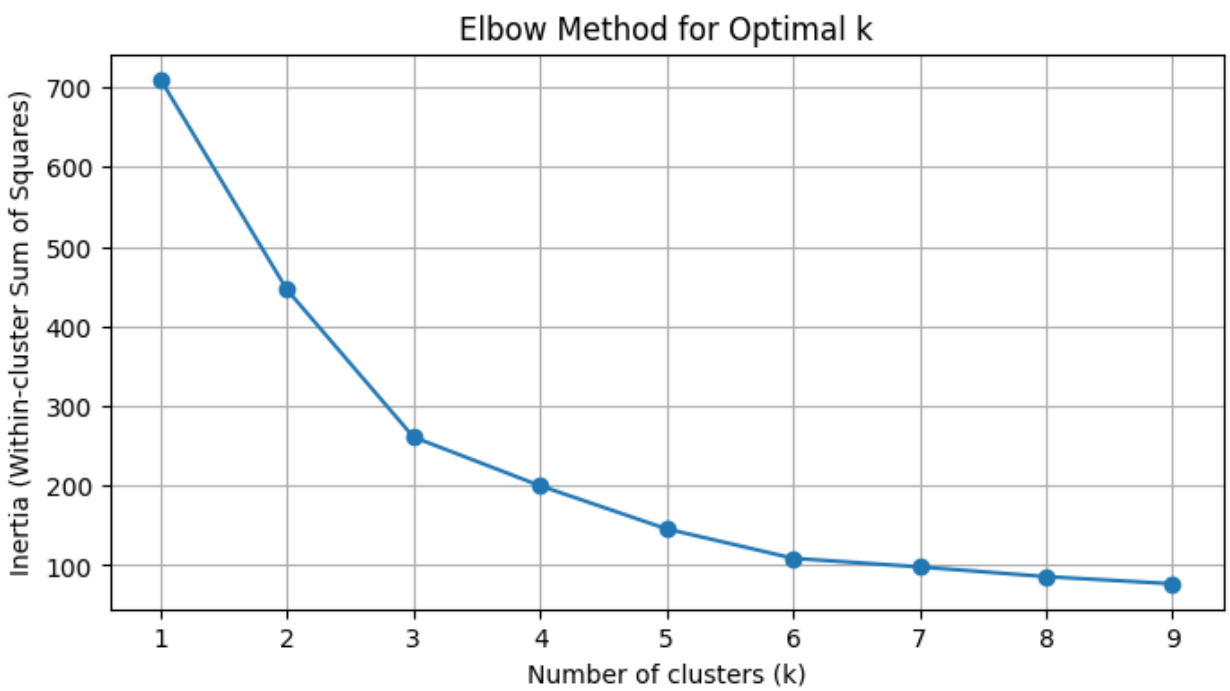
Output:


Rain vs. Temperature (Conditional Coloring)
Moderate Rain
Rather Heavy Rain
No Rain
Very Light Rain
Light Rain
Heavy Rain
Total Rain
Average Temperature
0
10
20
30
40
50
60
70
22
24
26
28
30
32
34

Code:

```
inertias = []
K_range = range(1, 10)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_norm)
    inertias.append(km.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(K_range, inertias, marker='o')
plt.xlabel('Number of clusters (k)')
plt.ylabel('Inertia (Within-cluster Sum of Squares)')
plt.title('Elbow Method for Optimal k')
plt.grid(True)
plt.show()
```

Output:



Code:

```
!pip install kneed
from kneed import KneeLocator
# Assuming you have 'inertias' and 'K_range' from your previous code
kl = KneeLocator(K_range, inertias, curve="convex",
direction="decreasing")
elbow_point = kl.elbow
print(f"Elbow point: {elbow_point}")
```

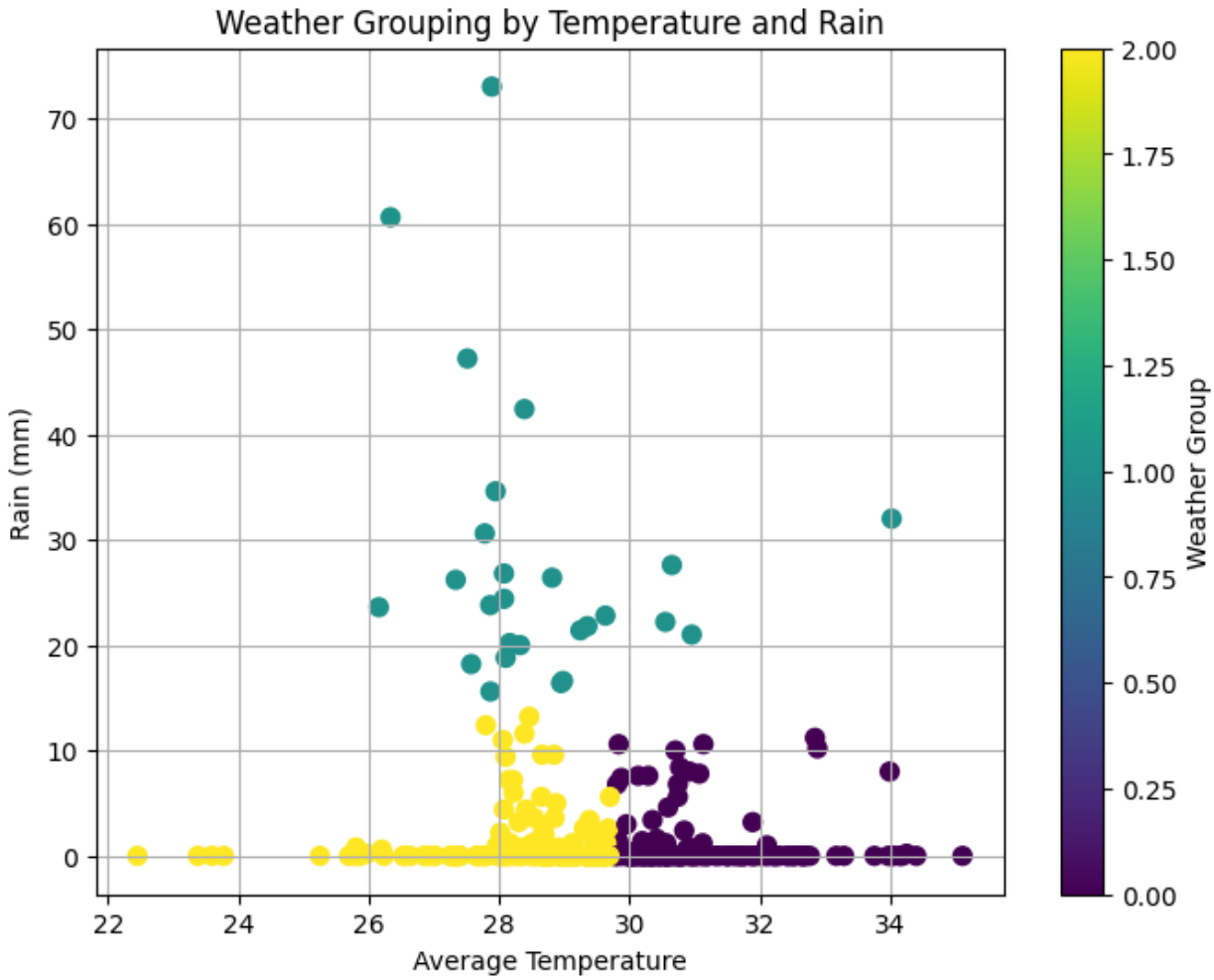
Output:

```
Requirement already satisfied: kneed in /usr/local/lib/python3.11/dist-packages (0.8.5)
Requirement already satisfied: numpy>=1.14.2 in /usr/local/lib/python3.11/dist-packages (from kneed) (2.0.2)
Requirement already satisfied: scipy>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from kneed) (1.15.2)
Elbow point: 3
```

Code:

```
from sklearn.cluster import KMeans

# Extract just the numeric features
X = fn[['average_temp', 'rain']]
mean_values = X.mean()
X = X.fillna(mean_values)
# Normalize (optional but helps clustering)
X_norm = (X - X.mean(axis=0)) / X.std(axis=0)

# Perform KMeans clustering
kmeans = KMeans(n_clusters=3, random_state=42)
clusters = kmeans.fit_predict(X_norm)

# Add the cluster labels to the original DataFrame
fn['weather_group'] = clusters

# Optional: Visualize the clustering result
plt.figure(figsize=(8, 6))
plt.scatter(fn['average_temp'], fn['rain'], c=clusters, cmap='viridis',
s=50)
plt.xlabel('Average Temperature')
plt.ylabel('Rain (mm)')
plt.title('Weather Grouping by Temperature and Rain')
plt.colorbar(label='Weather Group')
plt.grid(True)
plt.show()
```

Output:


Weather Grouping by Temperature and Rain
Rain (mm)
Average Temperature
Weather Group
70
60
50
40
30
20
10
0
22
24
26
28
30
32
34
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00

Code:

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

cluster_df = fn[['date', 'weather_group']].copy()
decreased_ranking_avg_with_cluster = pd.merge(decreased_ranking_avg,
cluster_df, on='date', how='inner')
increased_ranking_avg_with_cluster = pd.merge(increased_ranking_avg,
cluster_df, on='date', how='inner')

print("Average for Decreased Ranking:")
display(decreased_ranking_avg_with_cluster)
print("\nAverage for Increased Ranking:")
display(increased_ranking_avg_with_cluster)
```

Output:

```
Average for Decreased Ranking:
```

	date	average_temp	rain	danceability	energy	speechiness	acousticness	instrumentalness	liveness	valence	tempo	weather_group
0	2023-10-18	31.070833	7.8	0.000000	0.000000	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000	0
1	2023-10-19	29.104167	0.2	0.000000	0.000000	0.000000	0.000000	0.000000e+00	0.000000	0.000000	0.000000	2
2	2023-10-20	30.775000	8.4	0.609826	0.608957	0.066096	0.327463	5.008650e-03	0.164278	0.514217	137.102130	0
3	2023-10-21	27.512500	47.2	0.594400	0.624800	0.045487	0.257113	1.419487e-02	0.204013	0.483467	147.843867	1
4	2023-10-22	29.141667	0.0	0.633265	0.579088	0.059009	0.338616	1.335156e-02	0.171353	0.494588	131.699206	2
...	...	...	...	...	...	...	...	...	...	...	...	...
362	2024-10-14	34.181818	0.0	0.615187	0.654937	0.048469	0.205995	4.443884e-03	0.159875	0.457737	121.182875	0
363	2024-10-15	35.109091	0.0	0.629312	0.662625	0.053556	0.225386	5.631250e-07	0.149050	0.557500	109.766688	0
364	2024-10-16	30.363636	0.2	0.561450	0.616900	0.065615	0.290375	3.213500e-06	0.181795	0.458500	135.940350	0
365	2024-10-17	34.250000	0.2	0.575357	0.679643	0.051050	0.198557	7.305914e-04	0.180671	0.493129	137.391000	0
366	2024-10-18	34.022222	32.0	0.592500	0.668714	0.073407	0.284464	5.541286e-05	0.177879	0.503286	130.863714	1

367 rows × 12 columns

```
Average for Increased Ranking:
```

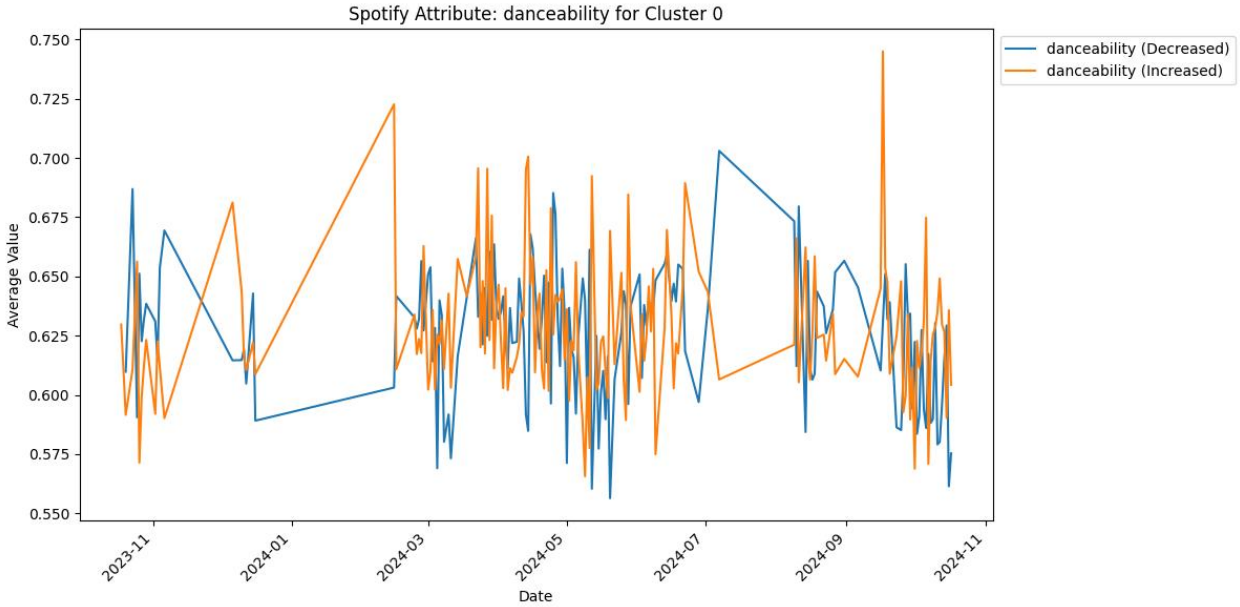
	date	average_temp	rain	danceability	energy	speechiness	acousticness	instrumentalness	liveness	valence	tempo	weather_group
0	2023-10-18	31.070833	7.8	0.629714	0.621939	0.064220	0.308918	0.012336	0.176563	0.513102	134.718694	0
1	2023-10-19	29.104167	0.2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	2
2	2023-10-20	30.775000	8.4	0.591583	0.632083	0.052400	0.361120	0.010617	0.249567	0.449000	130.832417	0
3	2023-10-21	27.512500	47.2	0.617913	0.588696	0.047583	0.355131	0.016380	0.168074	0.457957	128.497391	1
4	2023-10-22	29.141667	0.0	0.688833	0.709417	0.090808	0.218911	0.010685	0.195775	0.592833	122.515583	2
...	...	...	...	...	...	...	...	...	...	...	...	...
362	2024-10-14	34.181818	0.0	0.626700	0.681300	0.083710	0.197655	0.000003	0.202815	0.578200	118.136150	0
363	2024-10-15	35.109091	0.0	0.590389	0.695222	0.087094	0.259178	0.000797	0.220139	0.452711	130.556722	0
364	2024-10-16	30.363636	0.2	0.635778	0.736500	0.063683	0.167567	0.000608	0.165167	0.566211	113.000333	0
365	2024-10-17	34.250000	0.2	0.604286	0.696905	0.081695	0.258456	0.000037	0.143829	0.496952	119.901667	0
366	2024-10-18	34.022222	32.0	0.612957	0.637478	0.058109	0.283621	0.002645	0.167278	0.495261	116.785652	1

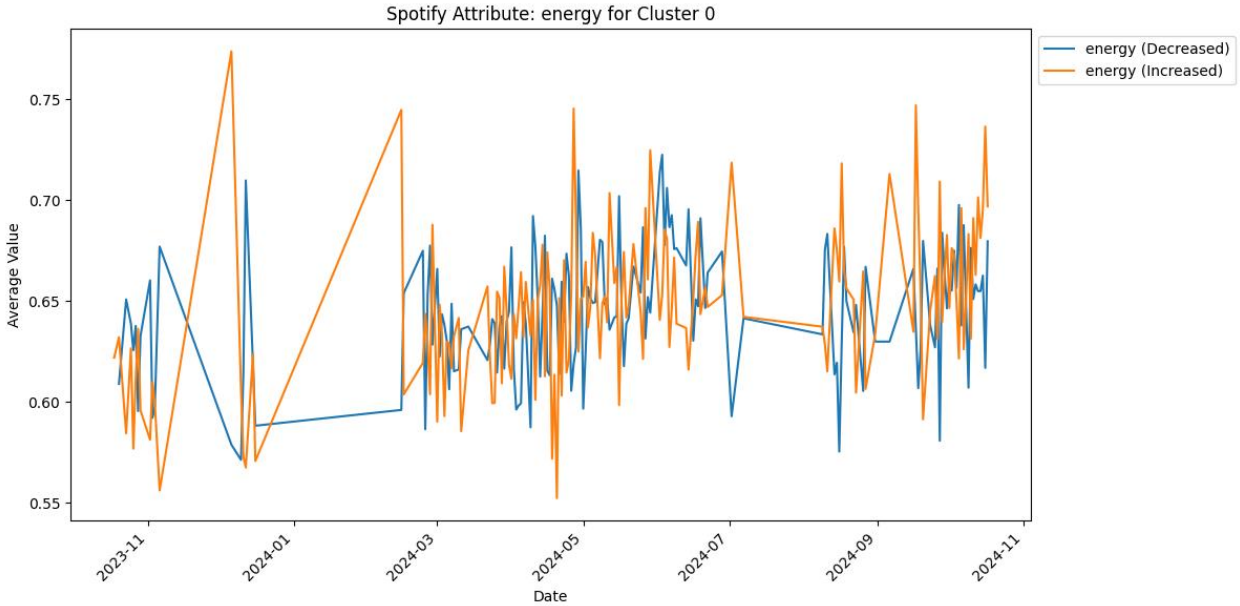
367 rows × 12 columns

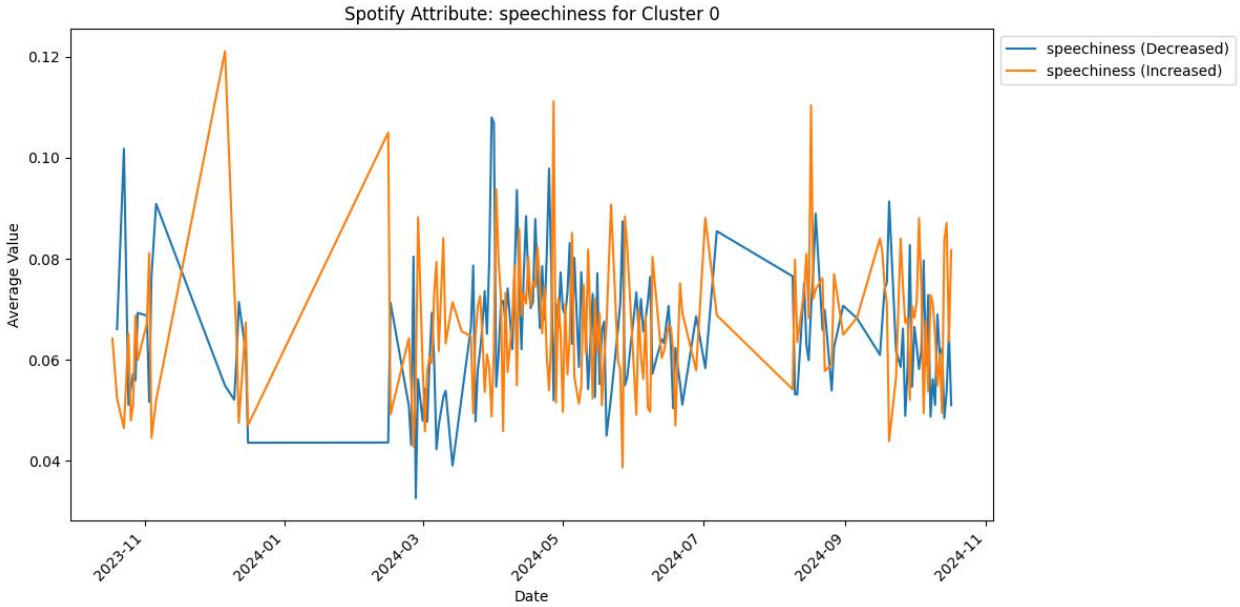
Code:

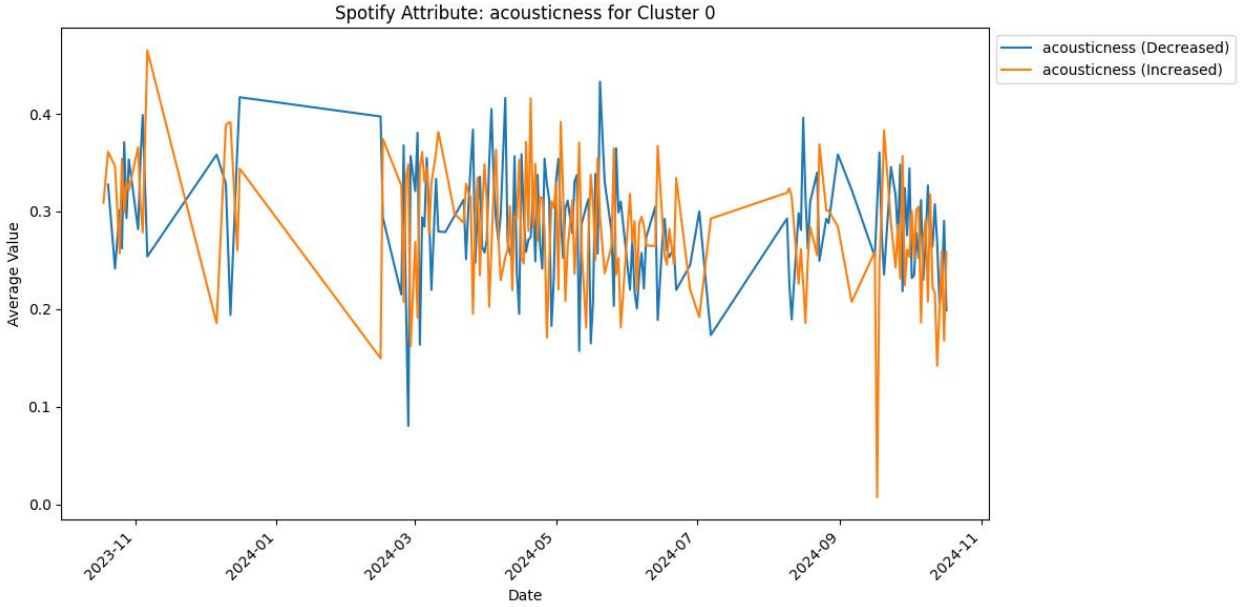
```
import matplotlib.pyplot as plt

# Assuming 'decreased_ranking_avg_with_cluster',
'increased_ranking_avg_with_cluster', and 'attributes' from your previous
code

for cluster in range(3):  # Iterate through clusters 0, 1, and 2
    for attribute in attributes:  # Iterate through each Spotify attribute
        fig, ax = plt.subplots(figsize=(12, 6))  # Create a new figure and
axes for each attribute

        # Filter data for the current cluster and attribute, skipping zero
values
        decreased_data_to_plot = decreased_ranking_avg_with_cluster[
            (decreased_ranking_avg_with_cluster['weather_group'] ==
cluster) & (decreased_ranking_avg_with_cluster[attribute] != 0)
        ]
        increased_data_to_plot = increased_ranking_avg_with_cluster[
            (increased_ranking_avg_with_cluster['weather_group'] ==
cluster) & (increased_ranking_avg_with_cluster[attribute] != 0)
        ]

        # Plot the attribute for decreased and increased rankings
        ax.plot(decreased_data_to_plot['date'],
decreased_data_to_plot[attribute], label=f'{attribute} (Decreased)')
        ax.plot(increased_data_to_plot['date'],
increased_data_to_plot[attribute], label=f'{attribute} (Increased)')

        # Customize the plot
        ax.set_xlabel('Date')
        ax.set_ylabel('Average Value')
        ax.set_title(f'Spotify Attribute: {attribute} for Cluster
{cluster}')  # Include attribute in title
        ax.legend(loc='upper left', bbox_to_anchor=(1, 1))
        plt.xticks(rotation=45, ha='right')
        plt.tight_layout()
        plt.show()
```

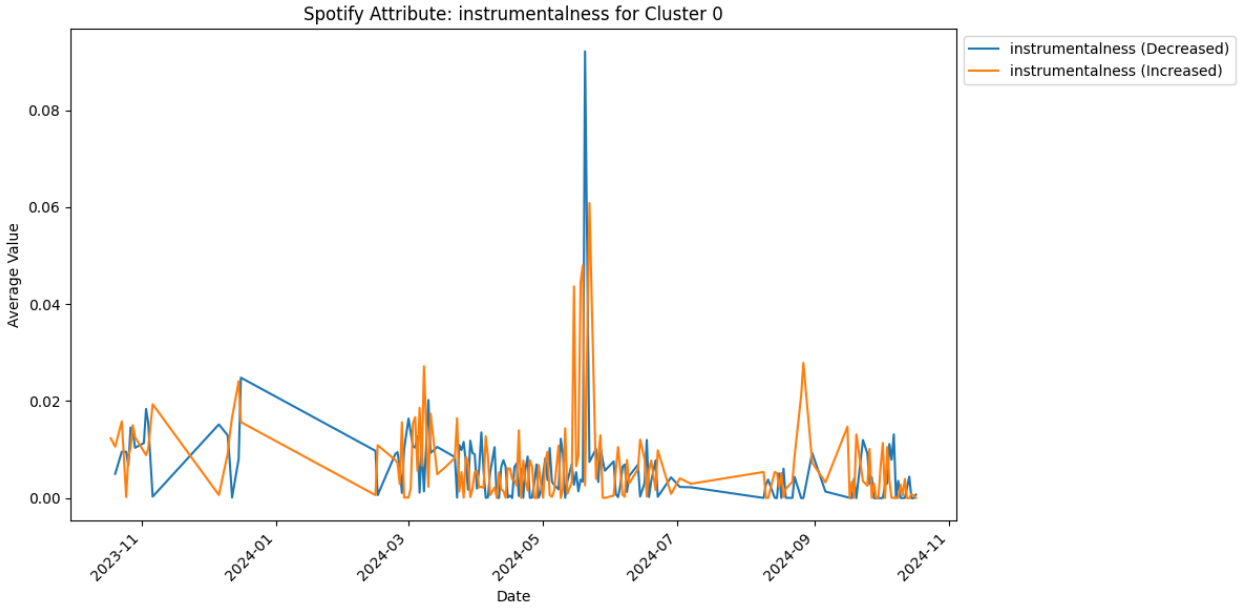
Output:

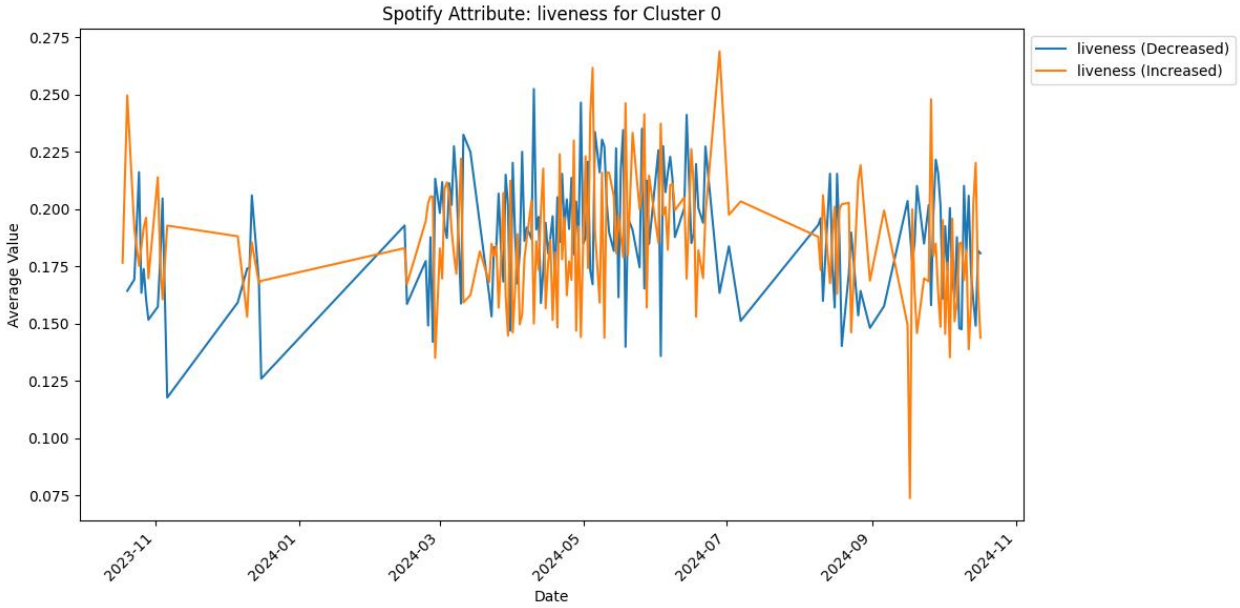

Spotify Attribute: danceability for Cluster 0
danceability (Decreased)
danceability (Increased)
Average Value
0.750
0.725
0.700
0.675
0.650
0.625
0.600
0.575
0.550
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

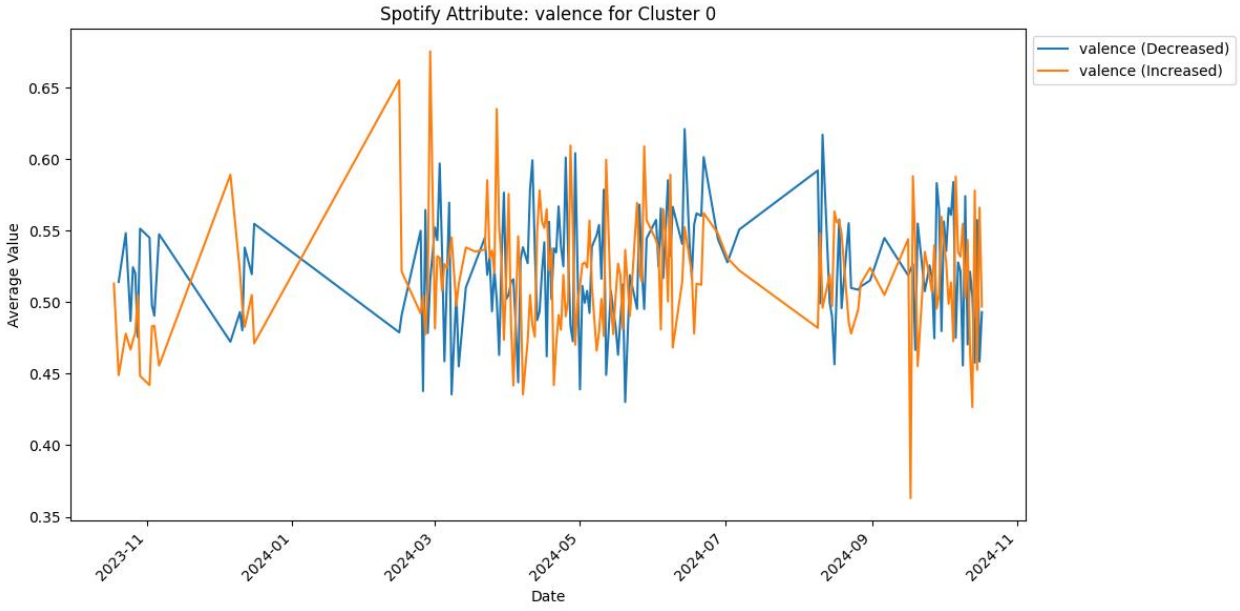

Spotify Attribute: energy for Cluster 0
energy (Decreased)
energy (Increased)
Average Value
0.75
0.70
0.65
0.60
0.55
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

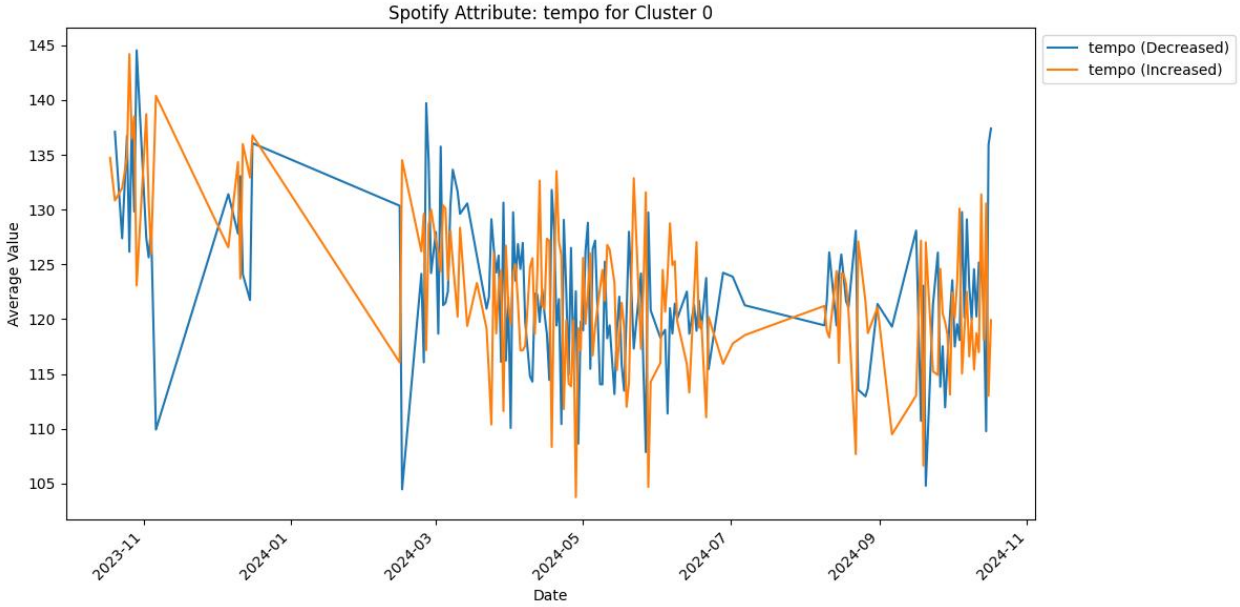

Spotify Attribute: speechiness for Cluster 0
speechiness (Decreased)
speechiness (Increased)
Average Value
0.12
0.10
0.08
0.06
0.04
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

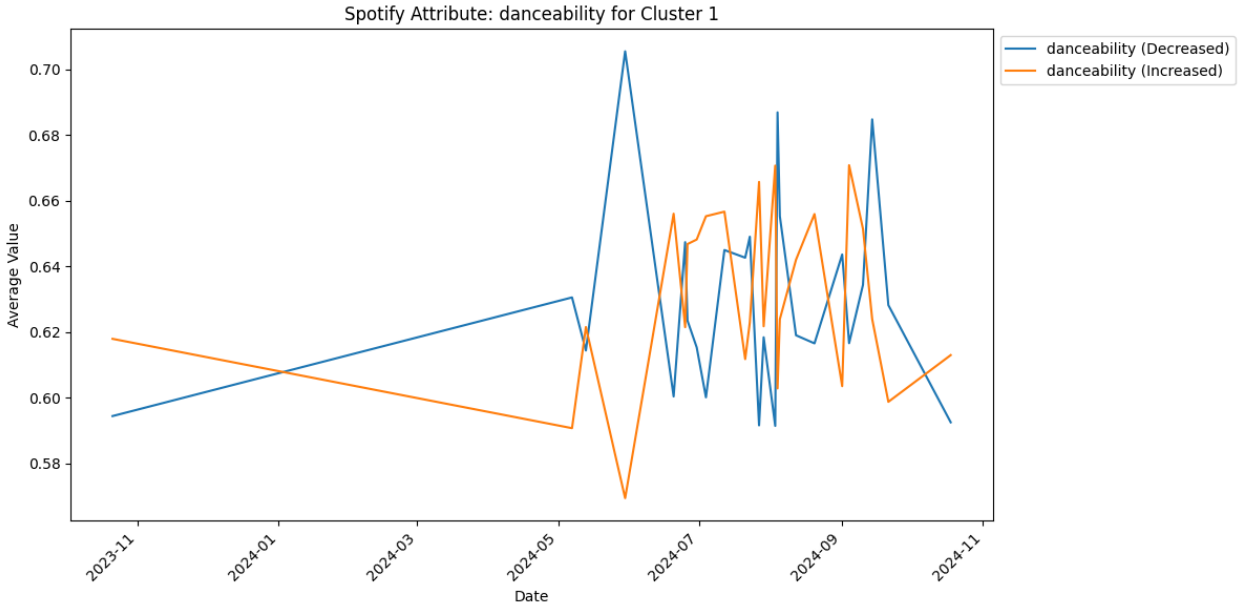

Spotify Attribute: acousticness for Cluster 0
acousticness (Decreased)
acousticness (Increased)
Average Value
0.4
0.3
0.2
0.1
0.0
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

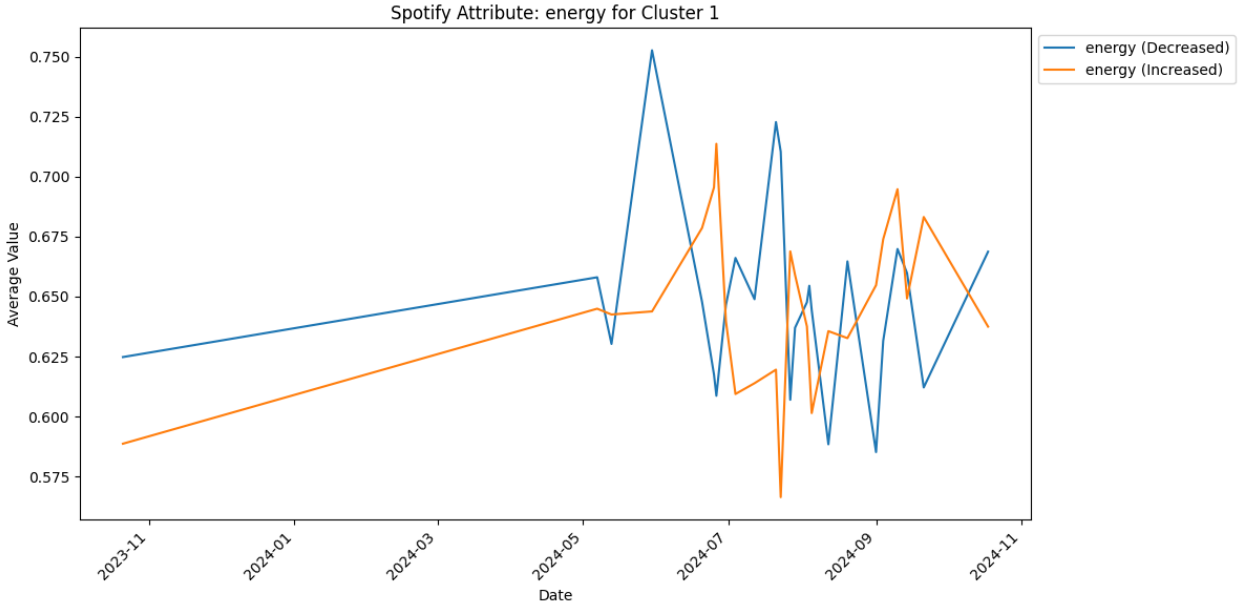

Spotify Attribute: instrumentalness for Cluster 0
instrumentalness (Decreased)
instrumentalness (Increased)
Average Value
0.08
0.06
0.04
0.02
0.00
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

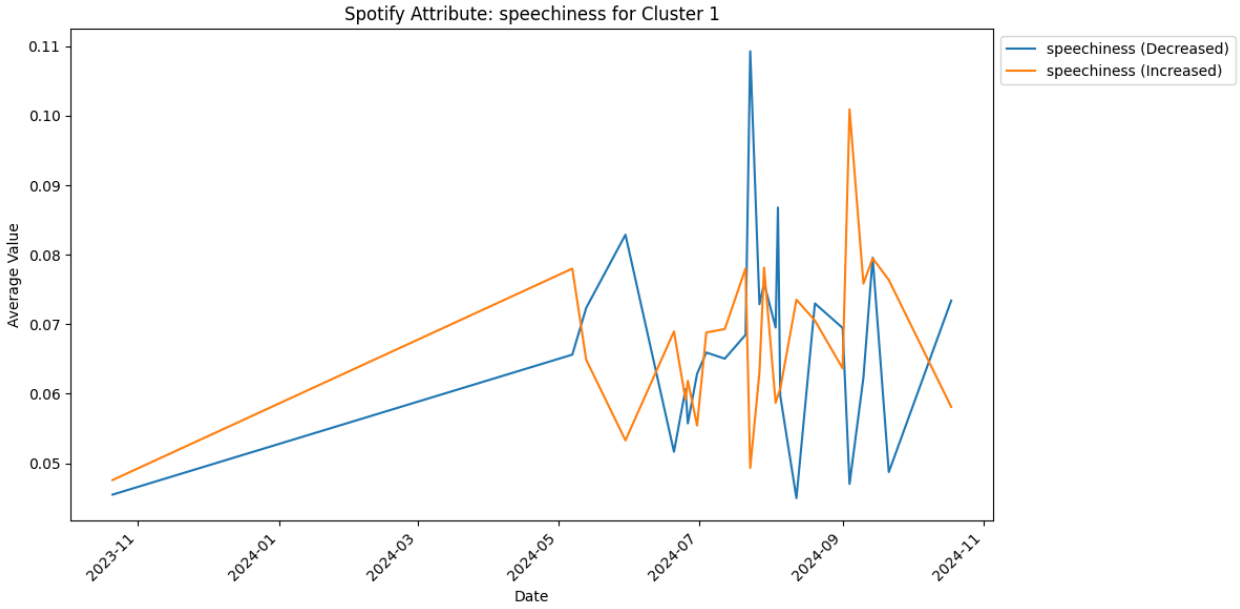

Spotify Attribute: liveness for Cluster 0
liveness (Decreased)
liveness (Increased)
Average Value
0.275
0.250
0.225
0.200
0.175
0.150
0.125
0.100
0.075
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

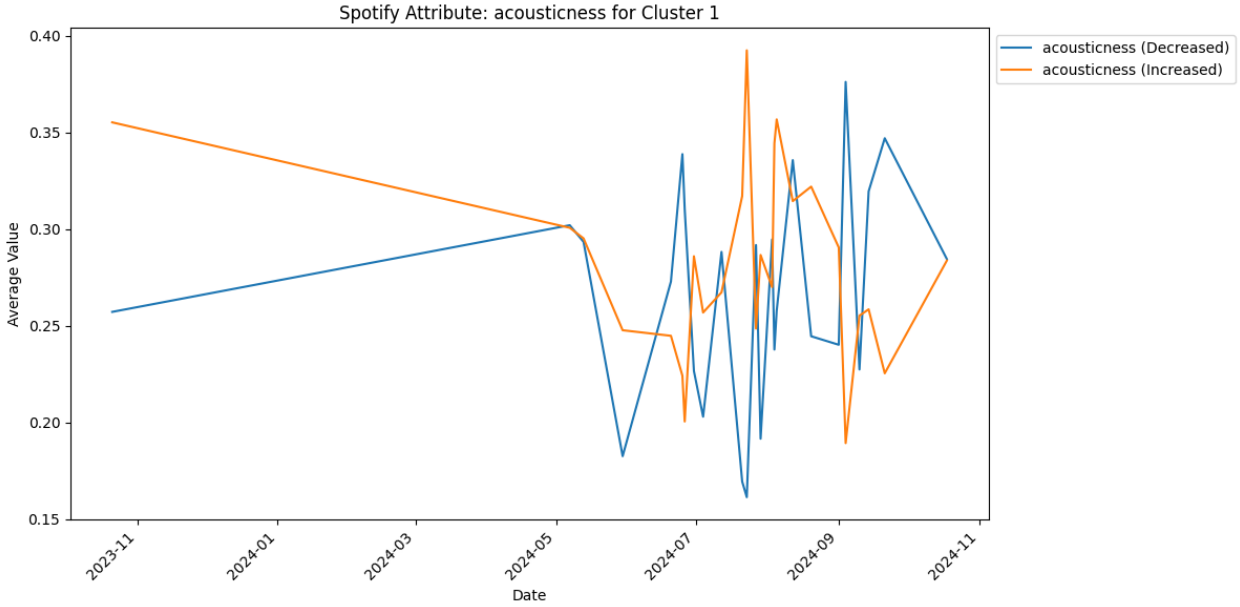

Spotify Attribute: valence for Cluster 0
valence (Decreased)
valence (Increased)
Average Value
0.65
0.60
0.55
0.50
0.45
0.40
0.35
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

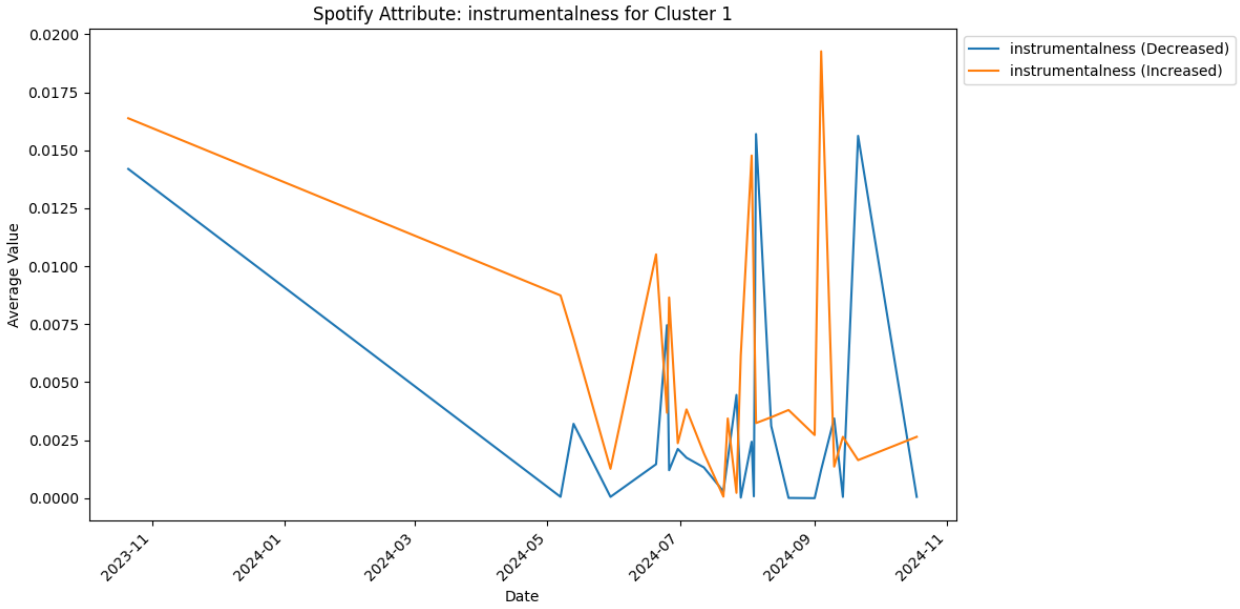

Spotify Attribute: tempo for Cluster 0
tempo (Decreased)
tempo (Increased)
Average Value
145
140
135
130
125
120
115
110
105
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

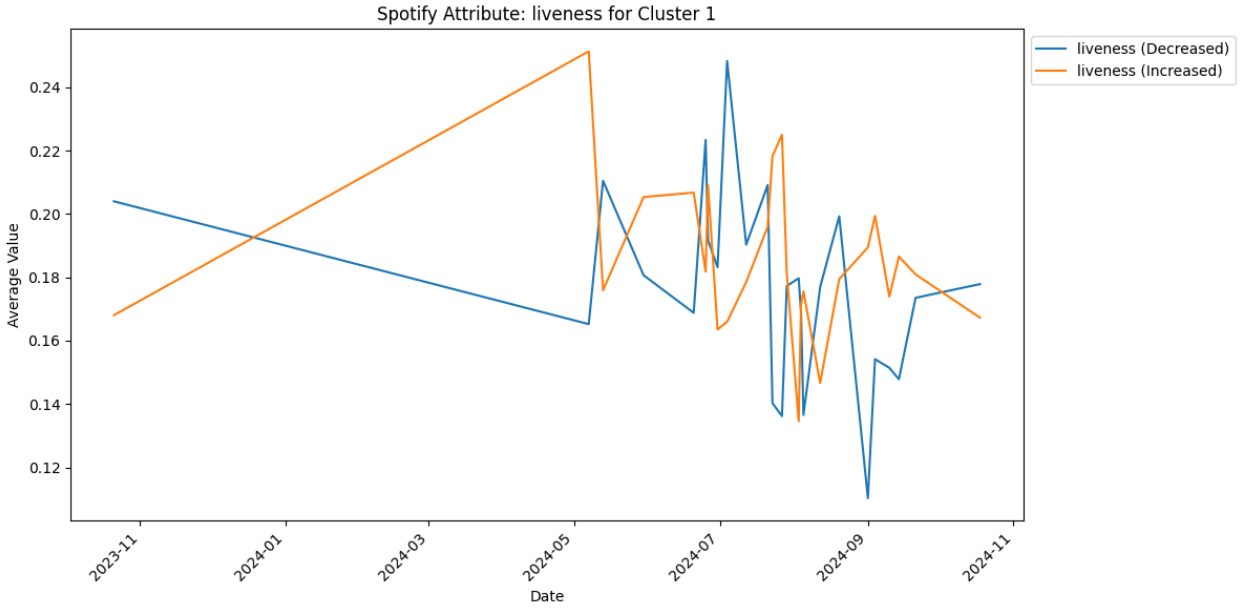

Spotify Attribute: danceability for Cluster 1
danceability (Decreased)
danceability (Increased)
Average Value
0.70
0.68
0.66
0.64
0.62
0.60
0.58
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

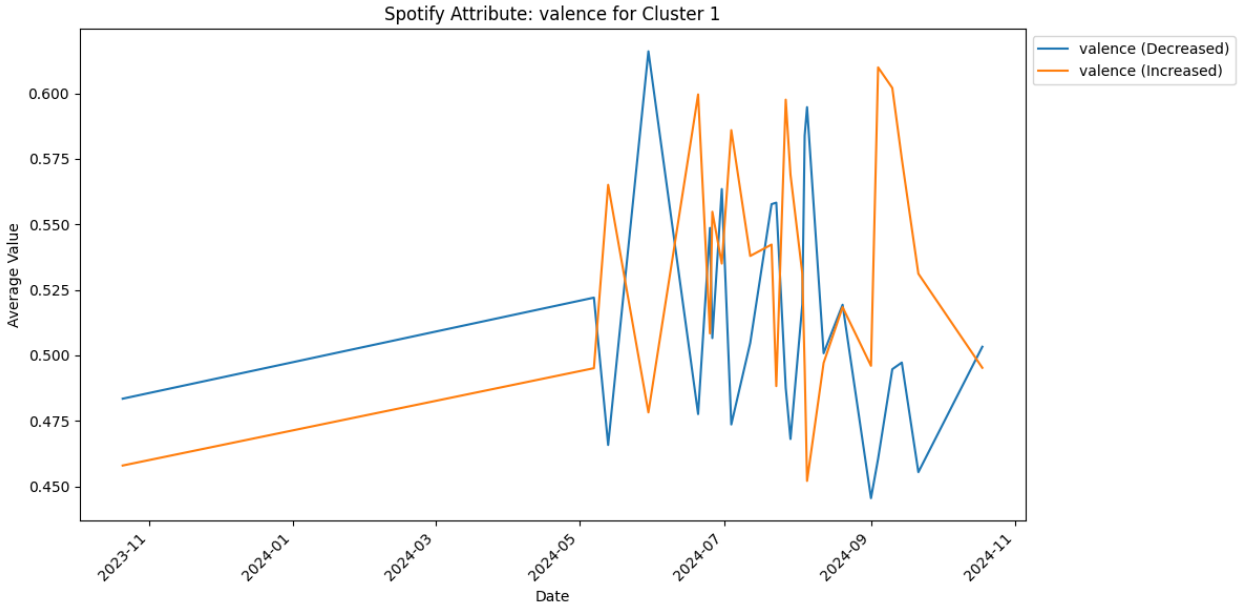

Spotify Attribute: energy for Cluster 1
energy (Decreased)
energy (Increased)
Average Value
0.750
0.725
0.700
0.675
0.650
0.625
0.600
0.575
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

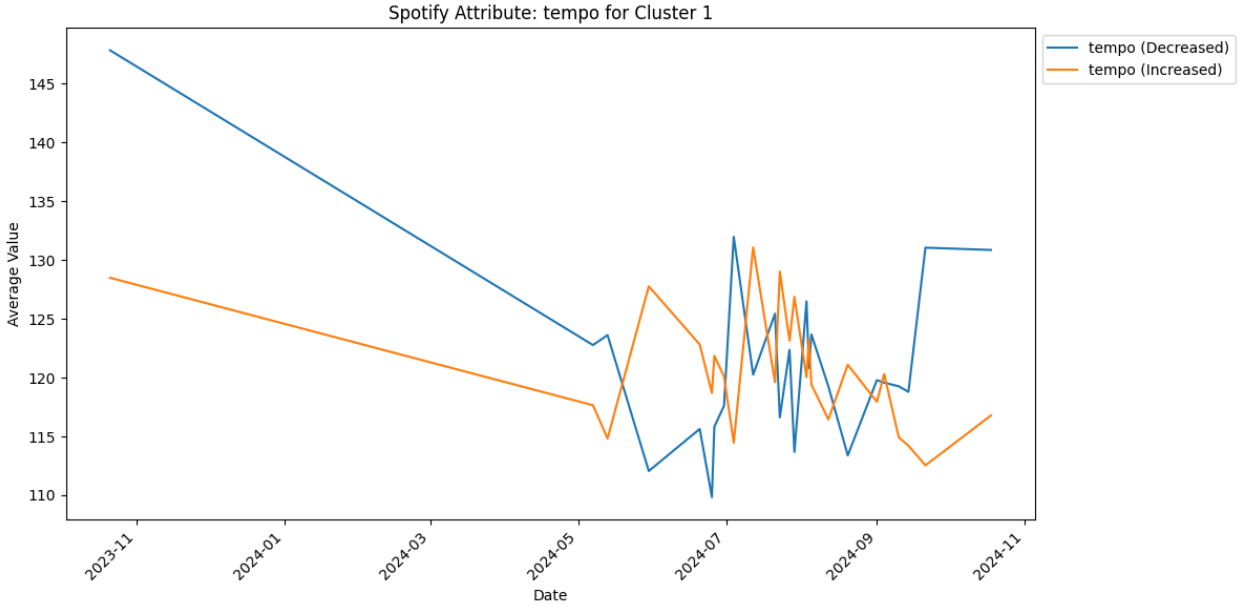

Spotify Attribute: speechiness for Cluster 1
speechiness (Decreased)
speechiness (Increased)
Average Value
0.11
0.10
0.09
0.08
0.07
0.06
0.05
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

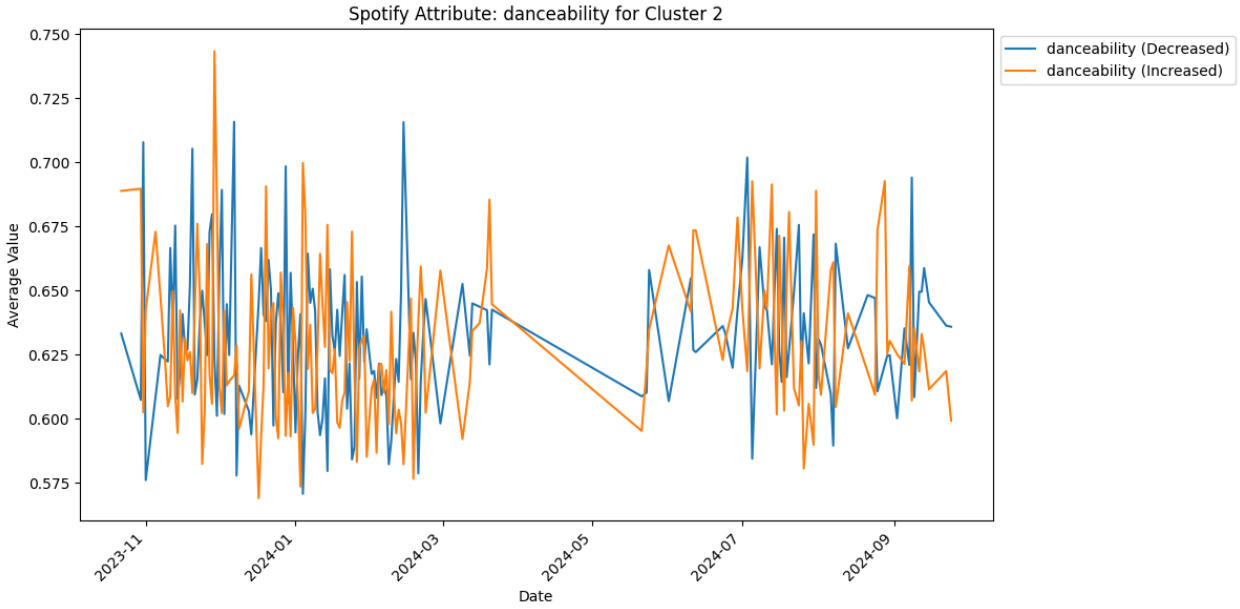

Spotify Attribute: acousticness for Cluster 1
acousticness (Decreased)
acousticness (Increased)
Average Value
0.40
0.35
0.30
0.25
0.20
0.15
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

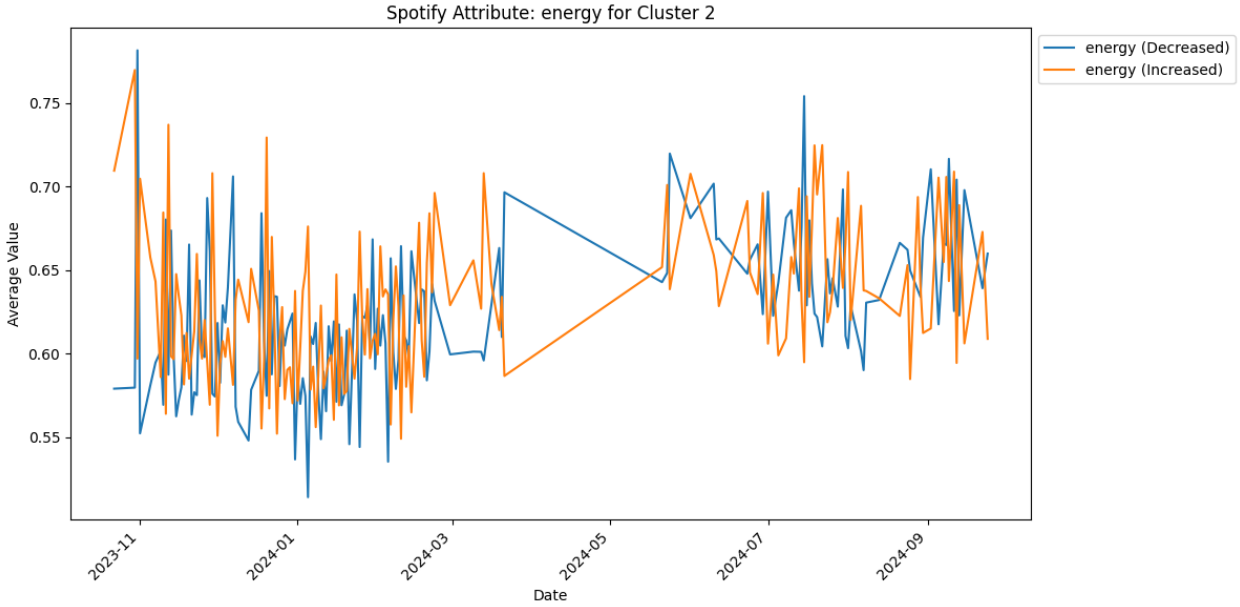

Spotify Attribute: instrumentalness for Cluster 1
instrumentalness (Decreased)
instrumentalness (Increased)
Average Value
0.0200
0.0175
0.0150
0.0125
0.0100
0.0075
0.0050
0.0025
0.0000
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

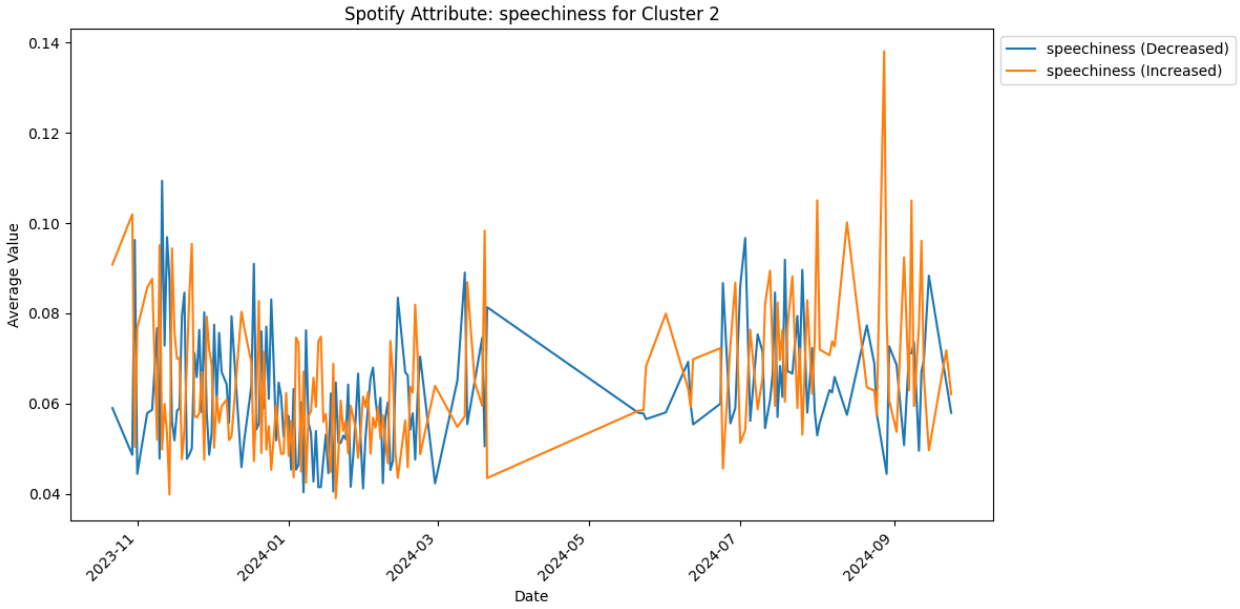

Spotify Attribute: liveness for Cluster 1
liveness (Decreased)
liveness (Increased)
Average Value
0.24
0.22
0.20
0.18
0.16
0.14
0.12
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

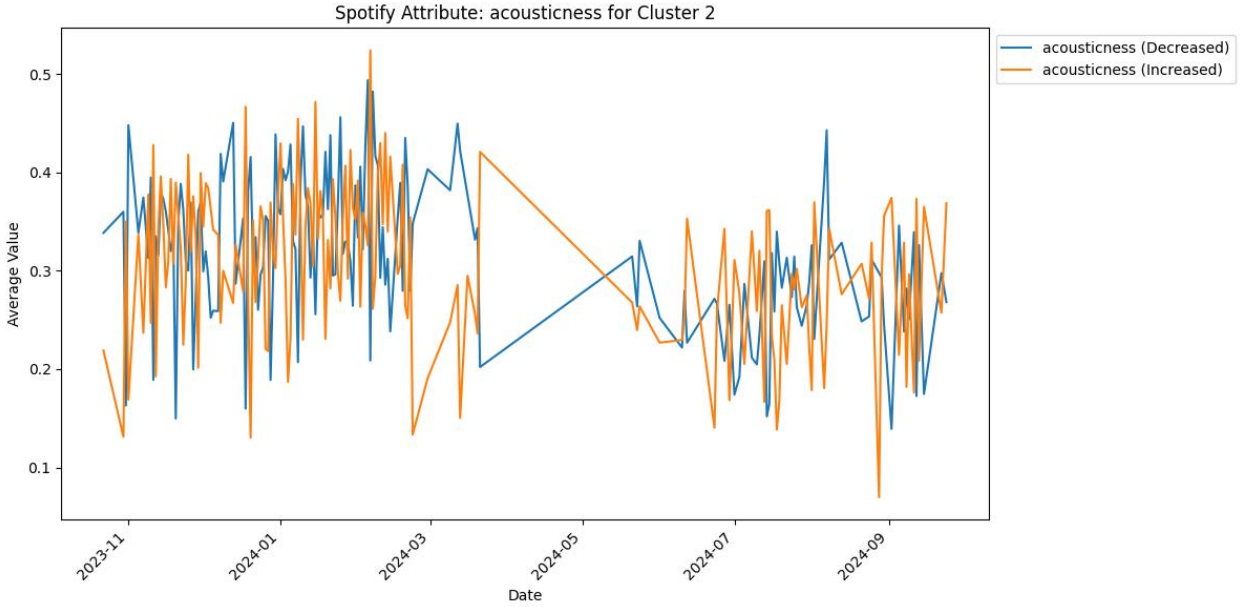

Spotify Attribute: valence for Cluster 1
valence (Decreased)
valence (Increased)
Average Value
0.600
0.575
0.550
0.525
0.500
0.475
0.450
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

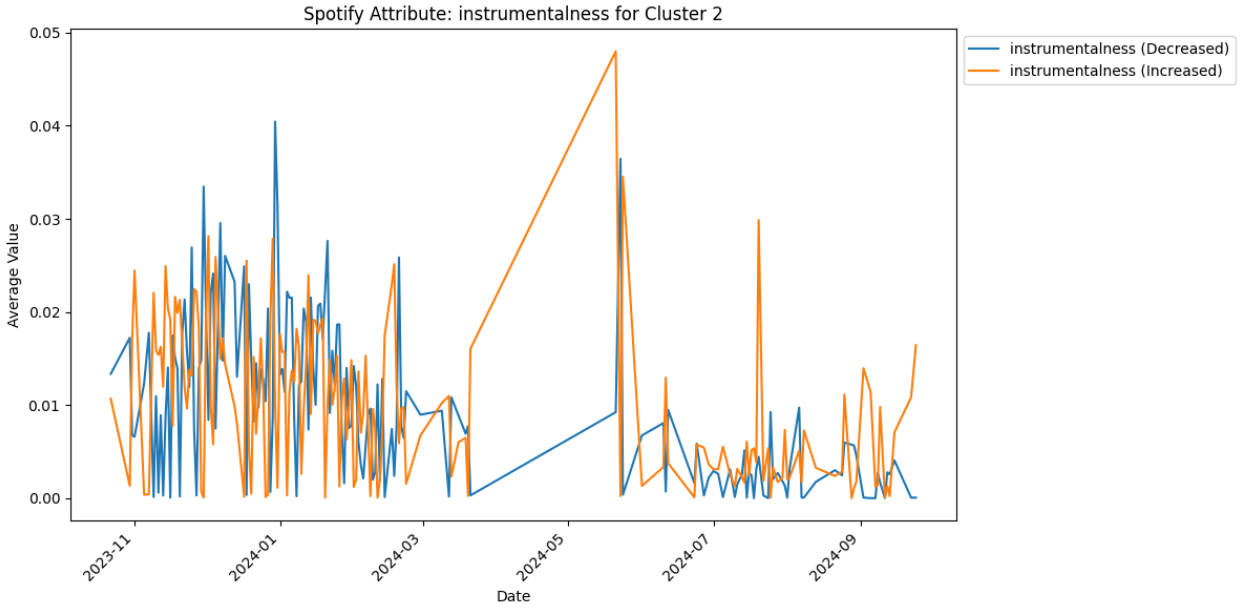

Spotify Attribute: tempo for Cluster 1
tempo (Decreased)
tempo (Increased)
Average Value
145
140
135
130
125
120
115
110
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
2024-11
Date

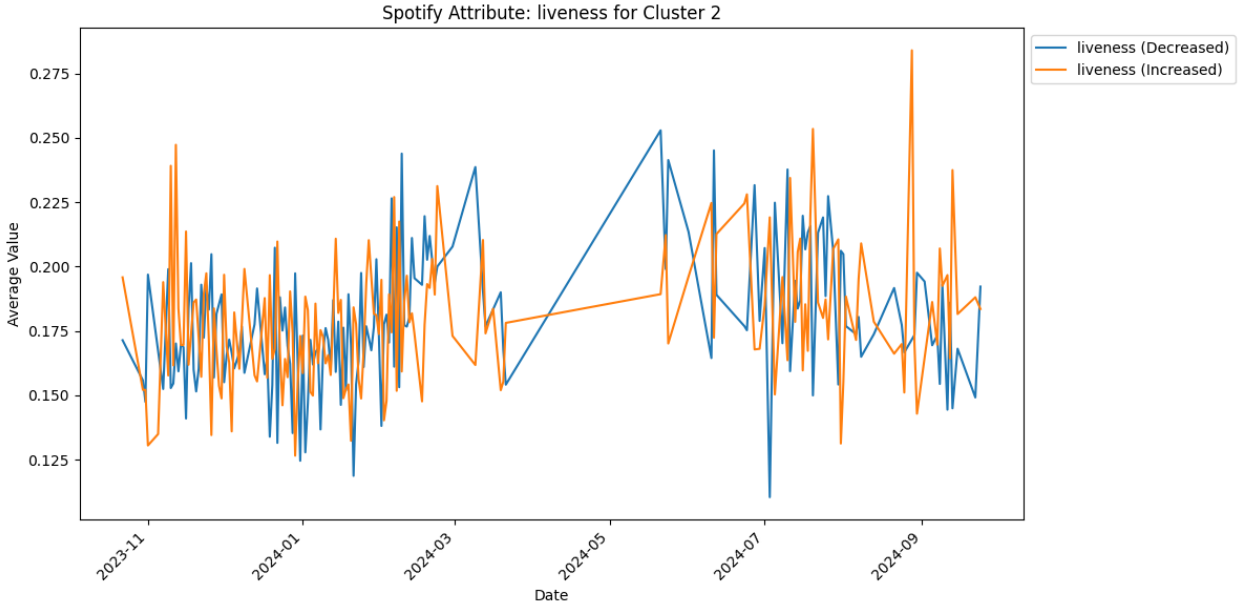

Spotify Attribute: danceability for Cluster 2
danceability (Decreased)
danceability (Increased)
Average Value
0.750
0.725
0.700
0.675
0.650
0.625
0.600
0.575
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date

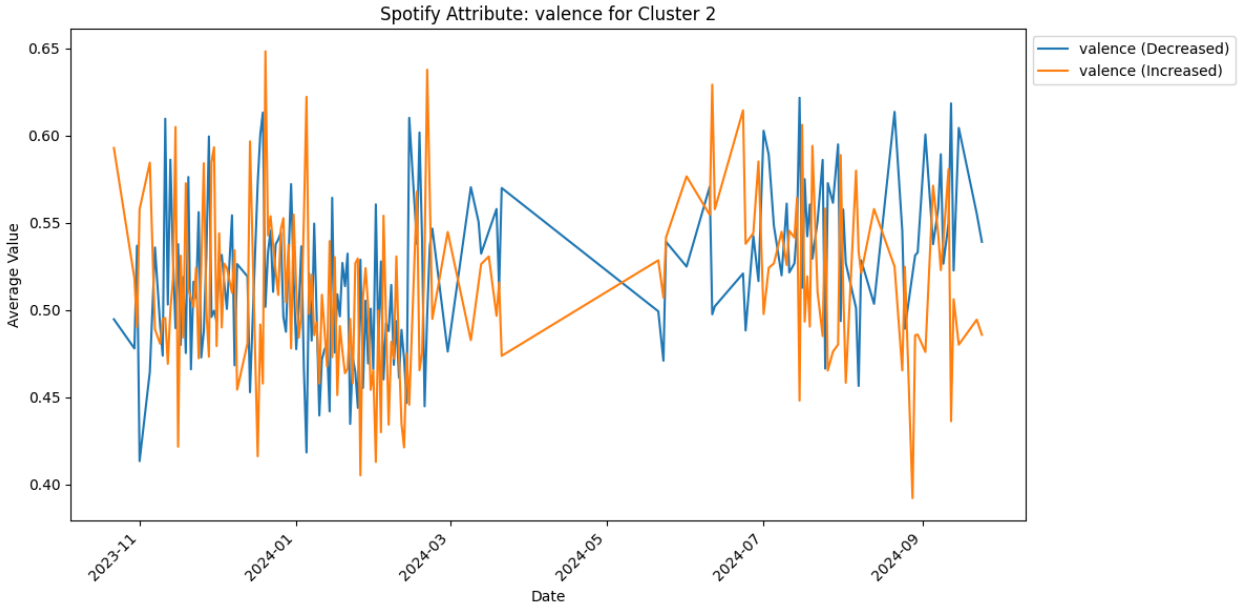

Spotify Attribute: energy for Cluster 2
energy (Decreased)
energy (Increased)
Average Value
0.75
0.70
0.65
0.60
0.55
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date

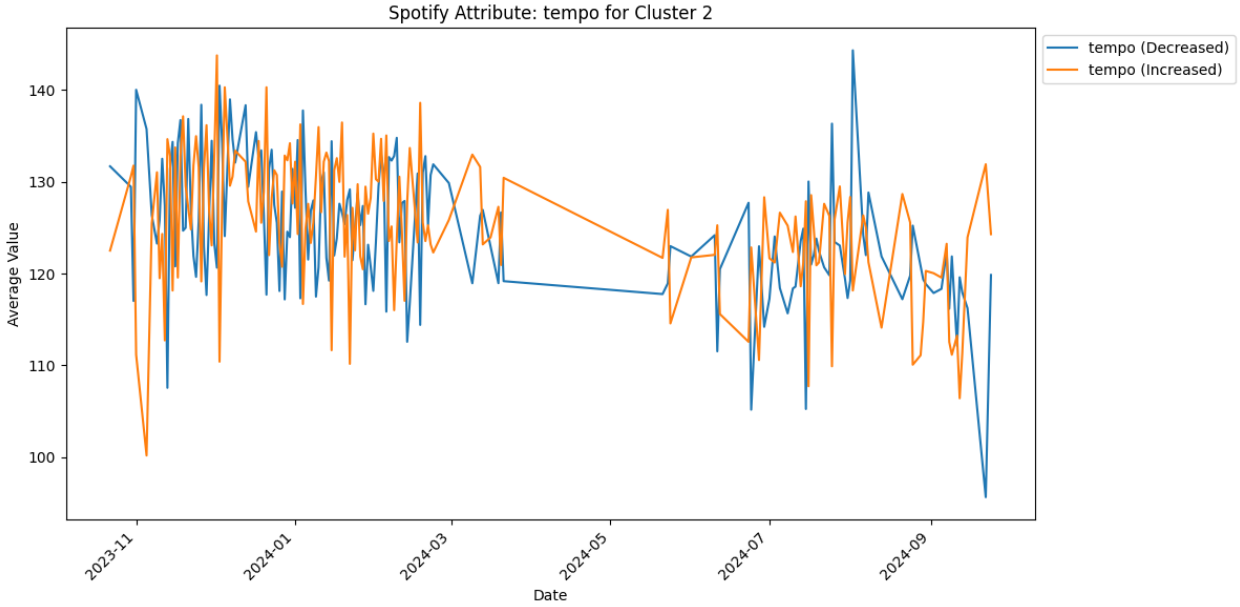

Spotify Attribute: speechiness for Cluster 2
speechiness (Decreased)
speechiness (Increased)
Average Value
0.14
0.12
0.10
0.08
0.06
0.04
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date


Spotify Attribute: acousticness for Cluster 2
acousticness (Decreased)
acousticness (Increased)
Average Value
0.5
0.4
0.3
0.2
0.1
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date


Spotify Attribute: instrumentalness for Cluster 2
instrumentalness (Decreased)
instrumentalness (Increased)
Average Value
0.05
0.04
0.03
0.02
0.01
0.00
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date


Spotify Attribute: liveness for Cluster 2
liveness (Decreased)
liveness (Increased)
Average Value
0.275
0.250
0.225
0.200
0.175
0.150
0.125
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date


Spotify Attribute: valence for Cluster 2
valence (Decreased)
valence (Increased)
Average Value
0.65
0.60
0.55
0.50
0.45
0.40
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date


Spotify Attribute: tempo for Cluster 2
tempo (Decreased)
tempo (Increased)
Average Value
140
130
120
110
100
2023-11
2024-01
2024-03
2024-05
2024-07
2024-09
Date

Machine Learning

Code:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

df = pd.read_csv('filtered_spotify_TH.csv')

temp = pd.read_csv('1023Tto1123T.csv')
rain = pd.read_csv('1023to1024.csv')

environment = pd.merge(temp, rain, on='date', how='inner')

columns_to_drop = ['spotify_id', 'daily_movement', 'weekly_movement',
'duration_ms', 'album_release_date', 'key',
                   'mode', 'daily_rank', 'time_signature', 'album_name',
'artists', 'popularity']
df = df.drop(columns=columns_to_drop)

spotify_TH = df[df['country'] == 'TH']

spotify_TH['snapshot_date'] = pd.to_datetime(spotify_TH['snapshot_date'],
errors='coerce').dt.strftime('%d/%m/%Y')
spotify_TH['snapshot_date'] =
spotify_TH['snapshot_date'].fillna('14/10/2024')

train = spotify_TH.merge(environment, left_on='snapshot_date',
right_on='date')
train.info()

train = train.drop(labels=['snapshot_date', 'date', 'country'], axis=1)
train.head()

test = (
   spotify_TH
   .drop(labels=['snapshot_date', 'country'], axis=1)   # ลบคอลัมน์ไม่จำเป็น
   .sort_values(by='name')                              # เรียงตามชื่อเพลง
```

```
    .drop_duplicates(subset='name')                          # ลบเพลงที่ชื่อซ้ำกัน
)

nameList = list(train.sort_values(by='name')['name'].unique())

train['is_explicit'] = train['is_explicit'].astype(int)  # False -> 0, True -
> 1

song_names = train['name']  # เก็บชื่อเพลงไว้
X = train.drop(['average_temp', 'rain', 'name'], axis=1)
y = train[['average_temp', 'rain']]

param_grid = {
    'estimator__n_estimators': [50, 100, 200],
    'estimator__max_depth': [None, 10, 20],
    'estimator__min_samples_split': [2, 5],
}

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)

base_model = RandomForestRegressor(random_state=42)
multioutput_model = MultiOutputRegressor(base_model)

grid_search = GridSearchCV(
    estimator=multioutput_model,
    param_grid=param_grid,
    scoring='r2',
    cv=3,
    verbose=1,
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

best_model = grid_search.best_estimator_
best_params = grid_search.best_params_

print("Best Parameters:", best_params)

y_pred = best_model.predict(X_test)
```

```python
X_result = X_test.reset_index(drop=True)
y_result = pd.DataFrame(y_pred, columns=['average_temp', 'rain'])

rounded_pred = np.round(y_pred)
rounded_true = np.round(y_test.to_numpy())

temp_acc = accuracy_score(rounded_true[:, 0], rounded_pred[:, 0])
rain_acc = accuracy_score(rounded_true[:, 1], rounded_pred[:, 1])

accuracy = (temp_acc + rain_acc) / 2

print(X_result.head())
print(y_result.head())
print(accuracy)
```

Output:

```
Fitting 3 folds for each of 18 candidates, totalling 54 fits
Best Parameters: {'estimator__max_depth': 10, 'estimator__min_samples_split': 5, 'estimator__n_estimators': 50}


(   is_explicit  danceability  energy  loudness  speechiness  acousticness  \
 0            1         0.894   0.725    -4.984       0.0432       0.00434
 1            0         0.631   0.822    -4.793       0.0269       0.21300
 2            0         0.770   0.677    -6.427       0.0589       0.52600
 3            0         0.711   0.809    -4.389       0.0955       0.04470
 4            0         0.633   0.607    -6.182       0.0532       0.18300

    instrumentalness  liveness  valence    tempo
 0          0.000012    0.0815    0.838  116.988
 1          0.001010    0.2120    0.879  149.886
 2          0.000483    0.1280    0.942  135.982
 3          0.000000    0.3390    0.816  106.017
 4          0.000001    0.1670    0.363   74.029  ,
    average_temp       rain
 0     33.503510  15.039359
 1     28.568992   1.630439
 2     28.512308   1.589733
 3     28.801291   1.109780
 4     28.410122   1.166777,
 0.14977307110438728)
```